

CSE 547Advanced Machine Learning Credit: 2.0

1. Mathematics of machine learning.
 2. Overview of supervised, unsupervised, and multi-task techniques.
 3. Advanced machine learning topics: Bayesian modeling and Gaussian processes, randomized methods, Bayesian neural networks, approximate inference, variational auto-encoders, generative models, applications.
 4. Applications of machine learning in natural language processing: recurrent neural networks, Backpropagation through time, long short-term memory, attention networks, memory networks, neural Turing machines, machine translation, question answering, speech recognition, syntactic and semantic parsing, GPU optimization for neural networks.
-

Mathematics of machine learning.

Important Concepts for Advanced Machine Learning

Variance vs Covariance vs Correlation

Variance measures the spread of a **single** dataset from its mean (how dispersed points are), while **covariance** measures how **two different variables** change together (their directional relationship). Variance is always positive, showing volatility (spread), whereas covariance can be positive (move together), negative (move opposite), or zero (no linear link). and **correlation** is standardized covariance, revealing both the strength and direction of a linear relationship, bound between -1 and +1.

ml **explainability** methods:

Sample Questions:

1. Define “learning” in the context of machine learning. How does it differ from human learning?
2. Explain why the term learning in “machine learning” is considered metaphorical.
3. What is the role of optimization in the learning process of a machine learning model?
4. Why can machine learning models not be said to “understand” the data they are trained on?
5. How does data quantity and quality affect the kind of “learning” achieved by a machine?
6. Suppose you train an image classifier to recognize fruits. After training, it performs well on apples and bananas but fails to recognize new images of mangoes. What does this reveal about the nature of “learning” in your model?
7. Consider a chatbot trained in millions of text conversations. Does it understand language? Justify your reasoning using the principles of machine learning.
8. A student argues that “since a machine can improve from experience, it learns just like a human.” How would you respond to this claim using technical reasoning?
9. Imagine a reinforcement learning agent that learns to play chess better than humans. Has it learned to understand chess strategy? Discuss.
10. Explain how overfitting and underfitting illustrate the limits of what machines can “learn.”
11. Compare and contrast human cognitive learning and machine learning from the perspectives of generalization, abstraction, and adaptability.

Does Machine Learning Actually Make Machines Learned?

The term *machine learning (ML)* suggests that machines are capable of “learning” from experience, much like humans. However, in reality, what machines do is fundamentally different from human or biological learning. Machine learning refers to a set of algorithms that enable systems to **improve their performance on a task by finding statistical patterns in data**, rather than through understanding or reasoning.

1. The Nature of Learning in Machines: In machine learning, “learning” is a **process of optimization**. A model adjusts its internal parameters (for instance, the *weights* in a neural network) to minimize an *error function* or *loss function*. This process improves its predictions on given data over time.

For example, in image classification, a model may “learn” that certain pixel patterns correspond to “cats” while others correspond to “dogs.” Yet, it doesn’t *understand* what a cat or dog is; it merely captures statistical correlations that help it distinguish between the two.

In contrast, **human learning** involves understanding, abstraction, reasoning, and the ability to generalize from very limited examples. A child who has seen only a few cats can recognize a new cat in a cartoon, sculpture, or even an abstract drawing, something most machine learning systems still struggle to do.

2. Why the Term “Learning” is Metaphorical: The use of the term *learning* in “machine learning” is largely metaphorical and mathematical. The “experience” for a machine is simply **data**, and “improvement” is quantified as a reduction in prediction error. Machines do not possess **awareness, comprehension, or intentionality**.

For example, when a language model predicts the next word in a sentence, it has no understanding of grammar or meaning; it simply relies on learned probabilities from large datasets. This reflects **pattern learning**, not **semantic learning**.

3. The Role of Optimization: All modern ML algorithms; from linear regression to deep neural networks; operate by **optimizing** an objective function. The system iteratively updates its parameters to minimize a defined error metric.

For instance:

- A neural network minimizes *cross-entropy loss* in classification tasks.
- A regression model minimizes *mean squared error (MSE)*.

This process is mechanical and data-driven, not conceptual. The machine does not know *why* an adjustment improves results; it simply follows gradient directions that reduce error.

4. Learning Patterns vs. Learning Meaning: Machine learning systems **learn patterns, not meanings**. They are excellent at **pattern recognition** but lack **semantic understanding** or **causal reasoning**.

Consider ChatGPT or similar large language models: they generate remarkably coherent text by predicting statistically likely word sequences but do not understand the meaning of those words. Similarly, an image classifier might confuse a dog with a wolf if both images share similar textures or backgrounds; revealing that it focuses on **correlated features** rather than conceptual categories.

5. Data Dependence and Generalization: Machine learning models are profoundly dependent on **large, high-quality, and diverse datasets** for effective training. The process of “learning” in these systems is essentially the discovery of statistical patterns and correlations within the training data. Consequently, the **scope and reliability of the model’s performance** are tightly bound to the **breadth and representativeness** of the data it has been exposed to. When the training data fails to capture the variability or complexity of real-world scenarios, the model’s predictions tend to deteriorate outside its learned context; a problem known as **poor generalization**.

For example, consider an image classification model designed to recognize different types of fruits. If it is trained primarily on images of apples, bananas, and oranges, it may achieve high accuracy within that limited dataset. However, when presented with an image of a mango fruit it has never encountered, the model may struggle to identify it correctly. This failure arises because the internal parameters (weights and feature representations) of the network have never been optimized for mango-like color, texture, or shape patterns. In such cases, the model’s **learning** is not genuine understanding but rather a memorization of visual correlations presents in the training data.

This limitation contrasts sharply with **human learning and reasoning**. Humans can generalize from a very small number of examples, often by drawing on prior knowledge, analogy, and contextual reasoning. For instance, a person who has never seen a mango before might still infer that it is a fruit by noticing shared characteristics such as the presence of a peel, sweet aroma, or seed; features abstracted from prior experiences with other fruits. Such cognitive generalization demonstrates a deeper, conceptual form of learning based on **semantic understanding and abstraction**, rather than raw pattern recognition.

The reliance of machine learning models on exhaustive and well-labeled datasets thus exposes their **domain-specific and narrow intelligence**. While they excel within the boundaries of their training distribution, their ability to adapt or extrapolate to new domains without retraining remains limited. This dependence on data not only restricts flexibility but also introduces challenges such as bias propagation, overfitting, and poor transferability. In essence, machine learning models “learn” only what they have statistically observed, whereas humans can infer beyond experience, adapt to unseen situations, and reason about unfamiliar phenomena.

6. Cognitive vs. Statistical Learning

Aspect	Human Learning	Machine Learning
Basis	Experience, reasoning, abstraction	Statistical pattern extraction
Data Requirement	Few examples	Large datasets
Understanding	Conceptual and semantic	Correlative and superficial
Generalization	Broad and flexible	Narrow and data-dependent
Error Correction	Reflective and intentional	Algorithmic optimization

This contrast emphasizes that machine learning is **syntactic** (structure-based) rather than **semantic** (meaning-based).

7. Philosophical and Practical Implications: The question of whether machines *learn* in the philosophical sense remains open. While reinforcement learning agents can surpass human performance in chess or Go, they do so through repeated optimization, not strategic understanding. They do not “know” what winning means; they simply maximize reward functions.

Thus, while the term “learning” is useful in describing the **functionality** of algorithms, it should not be confused with **cognition** or **conscious understanding**.

8. Limitations and Illustrative Examples:

- **Overfitting:** When a model memorizes training data without generalizing, it reveals that it hasn’t truly “learned” the underlying concept—only surface-level patterns.
 - **Chatbots and LLMs:** These systems generate contextually appropriate text but often fail in tasks requiring reasoning or real-world knowledge verification.
 - **Computer Vision:** Image classifiers can misclassify objects if lighting or orientation changes, indicating a lack of conceptual grasp.
-

9. Should “Learning” Be Redefined?

Some scholars argue that “learning” in AI should be viewed as *functional adaptation* rather than *understanding*. If we redefine “learning” to mean “improving performance through experience,” then yes—machines learn. But if we define learning as “understanding and reasoning about experience,” then machines do not.

This distinction matters ethically and philosophically because overestimating machine understanding could lead to misplaced trust, bias, or accountability issues.

Conclusion: Machine learning does not make machines “learned” in the human sense. Machines do not understand, reason, or possess awareness. They **optimize parameters, discover statistical patterns, and approximate mappings** from inputs to outputs. While these capabilities can mimic intelligent behavior, they are fundamentally different from human cognition.

In essence, **machine learning is powerful statistical inference, not comprehension**—a remarkable engineering achievement, but not genuine learning.

Mathematical Notation and Their Roles in Machine Learning

a, b, c – Scalar (integer or real)

A single numerical value representing quantities such as **learning rates**, **thresholds**, or **loss values**. Scalars form the basic building blocks of model parameters.

$\mathbf{x}, \mathbf{y}, \mathbf{z}$ – Vector (bold, lower case)

Ordered collections of numbers representing data samples, **feature vectors**, or **parameter sets**. Vectors serve as inputs to models and define parameter gradients in optimization.

$\mathbf{A}, \mathbf{B}, \mathbf{C}$ – Matrix (bold, upper case)

Two-dimensional numerical arrays used to represent datasets, linear transformations, and weight matrices in neural networks. Matrices support operations such as multiplication central to model computation.

$\mathbf{A}, \mathbf{B}, \mathbf{C}$ – Tensor (bold, upper case)

Multi-dimensional generalizations of matrices frequently used to store images, sequences, or batches of data. Tensors enable efficient computation across deep-learning architectures.

X, Y, Z – Random variable (normal upper case)

Variables representing uncertain quantities such as sampled data, model predictions, or noise. Random variables support the probabilistic formulation of learning algorithms.

$a \in \mathcal{A}$ – Set membership

Indicates that an element belongs to a set, useful in defining hypothesis spaces, label sets, or sample spaces.

$|\mathcal{A}|$ – Cardinality of a set $\mathcal{A}$

Number of elements in a set, often used to quantify dataset size, class counts, or model complexity.

$\|\mathbf{v}\|$ – Norm of vector $\mathbf{v}$

A measure of vector magnitude used in regularization, distance metrics, and gradient normalization.

$\mathbf{u} \cdot \mathbf{v}$ – Dot product of vectors $\mathbf{u}$ and $\mathbf{v}$

A scalar quantifying similarity or projection, fundamental in similarity measures, kernels, and geometric interpretations of data.

$\mathbb{R}$ – Set of real numbers

Represents continuous values used to define domains of parameters, inputs, or outputs.

$\mathbb{R}^n$ – Real vector space of dimension n

The space in which n -dimensional feature vectors or parameter vectors reside.

$\mathbf{y} = \mathbf{f}(\mathbf{x})$ or $\mathbf{x} \mapsto \mathbf{f}(\mathbf{x})$ – Function (map)

Describes a mapping from input to output, foundational for defining machine-learning models.

$\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}$ – Function mapping an n -dimensional vector to a scalar

A formulation used for loss functions, scoring functions, and regression models.

$\mathbf{A} \odot \mathbf{B}$ – Element-wise product of matrices $\mathbf{A}$ and $\mathbf{B}$

Component-wise multiplication used in gating mechanisms, feature masking, and attention mechanisms.

$\mathbf{A}^\dagger$ – Pseudo-inverse of matrix $\mathbf{A}$

Generalized inverse used in solving least-squares problems, particularly in linear regression with non-square or rank-deficient matrices.

$d^n \mathbf{f} / d\mathbf{x}^n$ – n -th derivative of $\mathbf{f}$ with respect to $\mathbf{x}$

Higher-order derivatives representing curvature properties, used in optimization algorithms and theoretical analysis of model behavior.

$\nabla_{\mathbf{x}} \mathbf{f}(\mathbf{x})$ – Gradient of $\mathbf{f}$ with respect to $\mathbf{x}$

Vector of first-order partial derivatives essential for gradient-based optimization such as SGD and Adam.

$\mathbf{H} \mathbf{f}$ – Hessian matrix of $\mathbf{f}$

Matrix of second-order partial derivatives capturing curvature information; used in second-order optimization and uncertainty estimation.

$X \sim P$ – Random variable X has distribution P

Specifies probabilistic behavior of data or noise, defining assumptions in generative models and Bayesian methods.

$P(X|Y)$ – Probability of X given Y

Conditional probability central to probabilistic inference, Bayesian networks, and supervised learning.

$\mathcal{N}(\mu, \sigma^2)$ – Gaussian distribution

A common model for continuous data and noise, used in generative models, regularization, and likelihood estimation.

$\mathbb{E}_{x \sim P} [f(X)]$ – Expectation of $f(X)$ under distribution P

Represents average model behavior or risk; used in expected loss, risk minimization, and sampling-based training.

$\text{Var}(f(X))$ – Variance of $f(X)$

Measures spread of values, important in assessing model stability, uncertainty, and noise.

$\text{Cov}(f(X), g(Y))$ – Covariance

Quantifies joint variability, used in feature selection, dimensionality reduction, and multivariate distribution modeling.

$\text{corr}(X, Y)$ – Correlation coefficient

Normalized measure of linear dependence, used in feature relevance analysis and statistical learning.

$D_{\text{kl}}(P \parallel Q)$ – Kullback-Leibler divergence

Measures discrepancy between distributions, fundamental in variational inference, regularization in VAEs, and information-theoretic learning.

$CE(P, Q)$ – Cross-entropy for distributions P and Q

A loss function used in classification, measuring dissimilarity between true and predicted distributions.

Matrix vs. Tensor: Detailed Description with Examples

Matrix

A **matrix** is a two-dimensional numerical array represented as **A** with rows and columns. It is a second-order structured object that generalizes vectors to two dimensions. Formally, a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ contains m rows and n columns.

Matrices are used to represent:

- **Dataset samples** (rows) with **features** (columns).
- **Linear transformations** applied to vectors.
- **Weight matrices** in neural networks (e.g., fully connected layers).

Example of a Matrix

$$A = \begin{bmatrix} 1 & 4 & 5 \\ 2 & 0 & 3 \end{bmatrix}$$

This matrix has 2 rows and 3 columns (shape: 2×3). It may represent two data samples, each with three feature values.

Machine Learning Use

1. Linear Regression Weight Matrix

The weight vector **w** is often treated as a $1 \times n$ matrix.

Prediction is given by:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

2. Neural Network Layers

A layer with n inputs and m outputs uses a weight matrix:

$$W \in R^{m \times n}$$

Multiplying $\mathbf{W}\mathbf{x}$ applies a learned transformation.

Tensor

A **tensor** is a multi-dimensional numerical array that generalizes scalars (0D), vectors (1D), and matrices (2D) to any number of dimensions. The tensor of order k has k axes.

For example:

- Order 0 $\rightarrow$ scalar
- Order 1 $\rightarrow$ vector
- Order 2 $\rightarrow$ matrix
- Order 3 $\rightarrow$ tensor (e.g., color image)
- Order 4+ $\rightarrow$ batches of images, video, or high-dimensional data

Tensors underpin modern machine learning systems, especially deep learning, where data are naturally multi-dimensional.

Example of a Tensor

A **3D tensor** representing an RGB image of height 2 and width 2:

$$T = \begin{bmatrix} \begin{bmatrix} 255 & 128 \\ 255 & 128 \end{bmatrix}, \begin{bmatrix} 10 & 20 \\ 10 & 20 \end{bmatrix}, \begin{bmatrix} 60 & 90 \\ 60 & 90 \end{bmatrix} \end{bmatrix} \text{Shape: (3 channels} \times \text{2 height} \times \text{2 width).}$$

Higher-order Tensor Example

A batch of 32 images, each of size $224 \times 224 \times 3$, forms a **4D tensor**:

(32,224,224,3)

Machine Learning Use

1. **Convolutional Neural Networks (CNNs)**
Images are processed as 3D tensors $(H \times W \times C)$.
Filters are also 3D tensors, and convolution produces new tensors representing feature maps.
2. **Sequence Models**
A batch of sequences can be represented as a 3D tensor:

(batch size , sequence length , embedding dimension)

3. **Transformer Attention Mechanisms**
Attention scores are stored in 4D tensors:

(batch , heads , query length , key length)

4. **Video Processing**
Videos form 5D tensors:

(batch , frames , height , width , channels)

Key Differences Between Matrix and Tensor

Property	Matrix	Tensor
Dimensionality	Always 2D	Can be any number of dimensions

Property	Matrix	Tensor
Order (Rank)	2	k (k ≥ 0)
Representation	Rows × Columns	Axes × Dimensions
Use Cases	Linear algebra, tabular data, fully connected layers	Images, sequences, video, batches, deep-learning operations
Operations	Matrix multiplication, transpose, inverse	Tensor reshaping, broadcasting, multi-axis operations, convolutions

Illustrative Example Showing the Difference

Scenario: Processing an Image for a Neural Network

- If the image is flattened into a vector, a matrix can transform it:

$$y = Wx$$

Here, **W** is a **matrix (2D)** and x is a vector.

- If the image is kept in its natural structured form (height × width × channels), it is a **tensor (3D)**:

$$I \in R^{H \times W \times 3}$$

Convolution is performed using:

$$O_{i,j,k} = \sum_{c=1}^3 \square \sum_{u,v} \square I_{i+u,j+v,c} \cdot F_{u,v,c,k}$$

where **F** is a **4D tensor** of filters.

Interpretation

Matrices cannot express multi-dimensional relationships inherent in structured data like images. Tensors preserve spatial and channel correlations, enabling deep-learning models to capture patterns more effectively.

Summary

- A **matrix** is a special case of a **tensor** with exactly two dimensions.
- A **tensor** generalizes matrices to higher dimensions, enabling representation of complex structured data such as images, time series, videos, and batches.
- Modern machine learning, especially deep learning, relies heavily on tensors because of the inherently multi-dimensional nature of real-world data.

Why GPUs Are Better Than CPUs for Tensor Processing

GPUs outperform CPUs in tensor processing because they are architecturally optimized for **massively parallel numerical computation**, whereas CPUs are optimized for **general-purpose sequential tasks**. Tensor operations involve large multi-dimensional arrays, and most of their computations can be performed concurrently. This makes GPUs ideal for modern machine-learning workloads.

1. Architectural Parallelism

CPU Architecture

- Typically, 4–32 cores
- Each core is optimized for **low-latency**, **complex branching**, and **serial computation**
- Large caches and control logic emphasize correctness and task switching

GPU Architecture

- Hundreds to thousands of smaller, simpler cores (e.g., NVIDIA RTX 4090 has >16,000 cores)
- Cores are optimized for **SIMD (Single Instruction, Multiple Data)** parallel execution
- Designed originally for graphics shading: computing similar operations on many pixels at once

Implications for Tensor Processing

Tensors often involve:

- Matrix multiplications
- Convolutions
- Elementwise operations
 - all of which break into independent subtasks.

GPUs can perform **tens of thousands of operations simultaneously**, whereas CPUs perform **dozens at most**.

2. High Throughput vs. Low Latency

CPU

- Optimized to minimize delay (low latency) for each instruction
- Good for logic-heavy tasks requiring decision-making (branching)

GPU

- Optimized for maximizing the number of operations completed per second (high throughput)
- Perfect for workloads with uniform operations over large datasets (tensors)

Tensor Relevance

Matrix multiplication for layers in a neural network:

$$Y = WX$$

contains millions (or billions) of multiply-accumulate operations that can be parallelized. GPUs dominate because they maximize throughput.

3. Specialized Hardware: Tensor Cores & SIMD Units

Modern GPUs include **Tensor Cores** or specialized matrix-multiply units.

Tensor Cores (NVIDIA)

- Perform matrix-multiply-and-accumulate (MMAs) at extremely high speed
- Operate on FP16, BF16, INT8 for deep learning
- Perform operations like:

$$D = A \times B + C$$

in a *single instruction*

SIMD Units

GPUs execute the same instruction across multiple data elements simultaneously.

Why CPUs Cannot Match This

CPUs lack:

- Tensor-specific circuits
- SIMD width comparable to GPUs
- Warp/block scheduling hardware

Thus, GPUs achieve **orders-of-magnitude faster tensor operations**.

4. Memory Bandwidth and Data Movement

Tensor operations require moving massive amounts of data quickly.

GPU Memory Advantages

- Very high bandwidth (e.g., >1 TB/s on modern GPUs)
- Optimized for streaming large blocks of data into compute cores
- Fast on-chip shared memory for thread groups
- Coalesced memory access patterns for parallel reading

CPU Memory Constraints

- Lower memory bandwidth (~50–100 GB/s)
- Access is optimized for small, random memory loads
- Cache hierarchy supports general-purpose workloads, not high-volume streaming

Machine Learning Impact

Large tensors like:

- Image batches
 - Transformer attention matrices
 - CNN feature maps
- require **fast sustained memory throughput**, which GPUs provide better than CPUs.
-

5. Scaling and Parallel Workload Distribution

GPUs allow:

- Thousands of parallel threads
- Thread-block scheduling with minimal overhead
- Streaming multiprocessors processing independent tensor chunks

Example: **Transformer Attention**

The attention matrix is:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Computations of $\mathbf{QK}^T$, softmax, and V-multiplication can be parallelized across:

- heads
- queries
- keys
- batch elements

GPUs distribute these parallel tasks automatically.

6. Tensor Framework Integration

Deep-learning libraries are GPU-optimized:

- PyTorch
- TensorFlow
- JAX
- cuDNN
- CUTLASS

These frameworks use:

- Kernel fusion
- GPU-optimized matrix multipliers
- Parallel convolution implementations
- Automatic gradient computation optimized for GPUs

CPU versions exist but lack specialized backend kernels.

Example: Matrix Multiplication CPU vs GPU

CPU

To compute:

$$C = A \times B$$

The CPU typically:

- Iterates through loops
- Computes sequentially or with limited multi-threading

GPU

The same operation is decomposed into thousands of block computations:

- Each thread computes one or a few elements
- Shared memory stores tiles of A and B
- Tensor Cores accelerate the multiply-accumulate operation

This enables **orders-of-magnitude speed increases**.

Summary of Why GPUs Excel at Tensor Processing

Reason

Massive parallelism

High throughput design

Specialized tensor hardware

High memory bandwidth

Efficient thread scheduling

Optimized ML libraries

Explanation

Thousands of cores vs. tens on CPUs

Optimized for simultaneous operations

Tensor Cores, SIMD-wide ALUs

Supports large tensor movement

Handles parallel tasks with low overhead

GPU kernels outperform CPU kernels

Summary

GPUs excel at tensor processing because their architecture is built for parallel, high-throughput, matrix-heavy computation, which aligns perfectly with the mathematical structure of tensor operations in machine learning.

Tensor Processing Unit (TPU)

A **Tensor Processing Unit (TPU)** is a specialized hardware accelerator designed specifically for **tensor-based machine learning computations**, especially deep learning. Architecturally and functionally, it sits **between GPUs and ASICs**, combining the flexibility of GPUs with the efficiency of purpose-built hardware.

TPU in Relation to CPU and GPU

CPU → General-Purpose Processor

- Optimized for sequential logic, branching, OS tasks
- Small number of powerful cores
- Good for control flow but slow for tensor-heavy computations

GPU → Parallel Numerical Processor

- Thousands of parallel cores
- Highly efficient for matrix multiplication and convolution
- Suited for deep learning training and inference
- Flexible and programmable through CUDA, PyTorch, TensorFlow, etc.

TPU → Domain-Specific ML Accelerator

- Designed *explicitly* for tensor operations, particularly matrix multiplies
- Extremely high throughput for deep-learning workloads
- Uses systolic array architecture for low-latency massive matrix operations
- Best suited for Google's ML ecosystem (TensorFlow, JAX)
- Achieves far higher matrix-multiply efficiency than GPUs for specific workloads

Hierarchy (General → Specialized)

CPU → GPU → TPU

- CPU: Most general
 - GPU: Specialized for parallel numerical operations
 - TPU: Hyper-specialized for tensor operations in machine learning
-

What Makes TPUs Unique?

Systolic Array Architecture

The core of a TPU uses a **large systolic array** (e.g., 256×256 or larger), where multiply–accumulate operations flow rhythmically through the array. This structure:

- Drastically reduces memory movement
- Maximizes usage of matrix multipliers
- Achieves extremely high TOPS (tera-operations per second)

GPUs use many SMs (streaming multiprocessors), but TPUs devote more real estate directly to matrix-multiply units.

High-Bandwidth Memory (HBM) and Dataflow Optimization

TPUs minimize memory bottlenecks by:

- Using high-bandwidth interconnects
- Operating on large blocks of tensor data at once
- Reducing DRAM loads through local buffer reuse

This is crucial for training large models such as BERT, GPT, and Vision Transformers.

Tensor-Centric Instruction Set

TPU instructions are minimal and specialized:

- Matrix multiply
- Convolution
- Nonlinear activation
- Reduction/aggregation

Unlike GPUs, TPUs do not optimize for rendering, graphics, or arbitrary compute. The narrow focus gives efficiency gains.

TPU Use Cases in ML Ecosystem

Training large-scale neural networks

TPUs excel at:

- Transformer models
- CNNs for vision
- RNN/LSTM-based sequence models
- Large batch training
- JAX-based parallel inference or training

Cloud providers (e.g., Google Cloud TPUs) deploy them for large-scale ML workloads.

JAX and TensorFlow Integration

Although TPUs support TensorFlow and JAX directly, PyTorch support exists but is indirect and less mature. This contrasts with GPUs which support:

- TensorFlow
 - PyTorch
 - JAX
 - Any CUDA-enabled framework
-

Performance Tier: Where TPU Stands

Training Speed

Generally:

TPU > High-End GPU > CPU

Inference Speed

Varies depending on model size and batch type:

- TPUs excel in large-batch inference
- GPUs sometimes perform better for small-batch or non-standard operations

Flexibility

GPU > TPU > CPU

TPUs sacrifice general-purpose programmability for speed.

Summary Table

Characteristic	CPU	GPU	TPU
Core Purpose	General-purpose compute	Parallel numerical compute	Tensor/matrix ML compute
Core Count	Few, powerful	Thousands	Large systolic arrays
Dataflow	Latency-optimized	Throughput-optimized	Tensor-optimized
Best For	Logic, OS tasks, small workloads	Deep-learning training & inference	Large-scale ML training, tensor-heavy workloads
Programmability	Very high	High	Medium (ML-focused)
Memory Bandwidth	Lowest	High	Very high
Efficiency for ML	Low	High	Extremely high

Conceptual Placement

A TPU is a **domain-specific super-accelerator** engineered to do one thing extremely well:
Perform tensor operations for deep learning with maximal efficiency.

It occupies a unique position:

- More specialized than a GPU
 - More flexible than fixed-function ASICs
 - Designed as a large-scale cloud accelerator rather than consumer hardware
-

Fixed-Function ASIC

ASIC = Application-Specific Integrated Circuit

A fixed-function ASIC is a hardware chip whose internal circuits are permanently wired to perform a dedicated computation. It **cannot be reprogrammed** to do a different task.

It is:

- **Application-specific** → built for one narrow task
- **Fixed function** → cannot be changed after manufacturing
- **Extremely fast and power-efficient** for that task
- **Not flexible** → cannot run arbitrary programs

This is in contrast to:

Hardware	Flexibility	Efficiency	Example Use
CPU	Very flexible	Lowest	Run general software

Hardware	Flexibility	Efficiency	Example Use
GPU	Moderately flexible	Very high	Matrix/tensor ops, graphics
TPU/ASIC	Not flexible	Highest	Specific AI ops (e.g., matmul)

Intuition

Think of a fixed-function **ASIC** like:

A specialized tool

- A **hammer** is designed specifically to hit nails.
- It is extremely efficient at that one job.
- You cannot use it to cut wood or tighten screws.

Whereas a CPU is a *Swiss army knife*: flexible but slower at any one thing.

Examples of Fixed-Function ASICs

1. Google TPU (partially ASIC-like)

- Optimized for matrix multiplication (common in neural networks)
- Cannot run general-purpose code

2. Bitcoin Mining ASICs

- Chips built *only* to compute SHA-256 hashing
- Thousands of times faster than CPUs/GPUs for BTC mining
- Useless for any other purpose

3. Network packet processors

- Hardwired for routing, encryption, checksum, etc.
- Very fast, low latency

4. Video codec chips

- Hardware H.264/HEVC encoding/decoding chips in smartphones
 - Much faster and more power-efficient than CPU/GPU encoding
-

How a Fixed-Function ASIC Differs from TPU/GPU

✓ GPU

- Programmable
- Runs many types of workloads
- Parallel but general-purpose

✓ TPU (Google)

- Designed specifically for tensor algebra
- Has matrix multipliers (systolic arrays)
- More ASIC-like than a GPU

✓ Fixed-Function ASIC

- *Even more specific* than TPU
- Hardwired for a **single computation**
- Cannot adapt to new algorithms or models

Example

Task	Best Hardware
Train deep learning model	GPU or TPU
Infer using a fixed AI model in production	ASIC
Decode video in real-time on a phone	ASIC
Solve arbitrary matrix multiplications	GPU/TPU

Analogy

Suppose you need to make a huge number of paper cuts:

- **CPU** = scissors → works, but slow
- **GPU** = 10 pairs of scissors at once → faster
- **TPU** = a machine that cuts 100 sheets at once → much faster
- **Fixed ASIC** = an automated cutter that cuts millions of sheets per second, but **ONLY** in one shape → extremely fast, but not reusable

Why Use Fixed-Function ASICs?

Advantages

- Ultra-low power use
- Fast processing
- High throughput
- Small physical size
- Ideal for mass deployment (phones, IoT, data centers)

Disadvantages

- Cannot be updated
 - Requires costly chip design
 - Bad for fast-changing fields (like ML research)
-

Real, concrete examples of *fixed-function ASICs* and *purpose-built accelerators* used by **NVIDIA, Google, Apple, and Tesla**, with what each chip does and why it matters.

NVIDIA — Fixed-Function ASICs + Accelerator Units

Although NVIDIA is known for GPUs, their chips contain **many fixed-function ASIC blocks** optimized for narrow tasks.

NVENC (NVIDIA Video Encoder ASIC)

- A fixed-function hardware block for H.264 / HEVC / AV1 video encoding
- Present in GeForce, Quadro, and Data Center GPUs

- **Cannot run arbitrary programs** → ONLY encodes video
- 10× more power-efficient than CPU encoding

Use case example:

OBS streaming, YouTube encoding, game capture.

RT Cores (Ray Tracing ASIC Block)

- Dedicated hardware for bounding volume hierarchy (BVH) traversal and ray-triangle intersections
- Introduced in NVIDIA RTX GPUs
- **Fixed function:** only performs ray intersection calculations

Why?

Ray tracing on general GPU cores is too slow.

Tensor Cores

- Specialized hardware for matrix multiply-and-accumulate (MMA)
- Accelerates FP16, INT8, INT4, TF32 operations
- **Semi-fixed:** optimized for tensor operations, not general-purpose computing

Use case:

AI training, inference, DLSS (deep learning super-sampling).

Google — TPUs & Specialized ASICs

Google is the biggest user of ASICs.

Google TPU (Tensor Processing Unit) — AI ASIC

- TPUv1–v4 are **application-specific integrated circuits**
- Purpose: accelerate neural network matmul operations
- Based on **systolic array** hardware
- Cannot run general-purpose workloads

Real use case:

Gmail autocomplete, Google Photos recognition, translation.

Google Video Transcoder ASIC

- Dedicated chip used in Google data centers
 - Accelerates AV1/VP9 video encoding
 - Far more efficient than GPU-based encoding
 - Powers YouTube video streaming infrastructure
-

Edge TPU (Coral AI)

- Ultra-small inference ASIC
 - Runs quantized ML models (INT8 only)
 - Used in IoT and embedded devices
-

Apple — Neural Engines & Fixed Multimedia ASICs

Apple designs extremely specialized silicon blocks in its A-series and M-series chips.

Apple Neural Engine (ANE)

- ASIC for neural network inference
- First introduced in A11 Bionic
- Up to 32 cores; does ~15 trillion ops/sec (varies by version)
- **Runs only specific ML operations** like:
 - convolutions
 - LSTMs
 - image segmentation
 - face recognition ops

Use case:

FaceID, Siri dictation, object detection in Photos.

Apple Image Signal Processor (ISP)

- Dedicated hardware for:
 - noise reduction
 - HDR fusion
 - autofocus
 - tone mapping
 - Essential for iPhone camera processing
 - Much faster than CPU/GPU implementation
-

Apple Video Codec ASIC

- Hardware decode/encode for:
 - H.264, HEVC, ProRes
 - Extremely power-efficient for 4K and 8K video recording
-

Tesla — FSD (Full Self-Driving) Chip

Tesla designs **vehicle-grade ASICs** specifically for autonomous driving.

Tesla Hardware 3.0 (FSD Computer)

- Contains **two custom Tesla-designed inference ASICs**
- Each chip:
 - 36 TOPS (trillion operations per second)
 - Optimized for **CNN-based perception workloads**
 - Runs neural networks for:
 - object detection
 - lane recognition
 - path planning

Key point:

The FSD ASIC is specialized for Tesla's driving neural networks — it cannot run YouTube, games, or general computation.

Tesla Dojo D1 Chip (Training ASIC)

- Purpose-built ASIC for **AI training** in Tesla data centers
- Tileable architecture enabling large-scale systolic arrays
- Designed to replace GPUs for specific NN training workloads
- High bandwidth + low power consumption

Use case:

Training Tesla's vision networks from driving footage.

Summary Table: Real Fixed-Function / Specialized Hardware

Company	Specialized Hardware	Type	Purpose
NVIDIA	NVENC, RT Cores, Tensor Cores	ASIC blocks	Video, ray tracing, AI
Google	TPU, Edge TPU, Video Transcoder	Full ASIC	AI & video infrastructure
Apple	Neural Engine, ISP, ProRes encoder	ASIC blocks	Mobile AI, camera, video
Tesla	FSD Chip, Dojo D1	Full ASIC	Self-driving inference + training

$\|\mathbf{v}\|$ – Norm of Vector $\mathbf{v}$

The norm of a vector $\mathbf{v}$ is a scalar quantity representing the magnitude or length of the vector in a given vector space. It provides a measure of how “large” the vector is and plays a foundational role in geometry, optimization, and machine-learning algorithms. Norms impose structure on vector spaces by defining distances, similarities, and regularization constraints on model parameters or data representations.

norm and **normalization** are related concepts, but they are **not the same**. A **norm** is a *function* that measures the **magnitude** or **length** of a vector. **Normalization** is a *process* that uses a norm to scale a vector. Example: **unit normalization** divides the vector

Common Types of Norms

1. ℓ_2 Norm (Euclidean Norm)

The ℓ_2 norm of a vector $\mathbf{v} = (v_1, v_2, \dots, v_n)$ is defined as:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

It measures the straight-line distance from the origin to the point represented by $\mathbf{v}$ in $\mathbb{R}^n$.

Example:

For $\mathbf{v} = (3, 4)$:

$$\|\mathbf{v}\|_2 = \sqrt{3^2 + 4^2} = 5.$$

Use in Machine Learning:

- Appears in L2 regularization (Ridge regression, weight decay in neural networks).
 - Encourages smaller parameter values, promoting generalization.
 - Defines Euclidean distance used in clustering (e.g., k-means) and nearest-neighbor methods.
-

2. ℓ_1 Norm (Manhattan Norm)

Defined as the sum of absolute values:

$$\| \mathbf{v} \|_1 = |v_1| + |v_2| + \dots + |v_n|.$$

Example:

For $\mathbf{v} = (3, -4)$:

$$\| \mathbf{v} \|_1 = |3| + |-4| = 7.$$

Use in Machine Learning:

- Applied in L1 regularization (Lasso regression).
 - Promotes sparsity, leading to models with fewer non-zero parameters.
 - Useful in feature selection and compressed sensing.
-

3. ℓ_∞ Norm (Max Norm)

Defined as the maximum absolute value among the components:

$$\| \mathbf{v} \|_\infty = \max_i |v_i|.$$

Example:

For $\mathbf{v} = (3, -4, 2)$:

$$\| \mathbf{v} \|_\infty = 4.$$

Use in Machine Learning:

- Relevant in optimization problems with box constraints (e.g., weight clipping in GANs).
 - Used in robustness analysis, such as ℓ_∞ -bounded adversarial attacks.
-

Role of Vector Norms in Machine Learning

1. Measuring Distance Between Samples

Norms define metrics used to compute distances:

$$d(x, y) = \|x - y\|.$$

This is central to clustering, dimensionality reduction, and nearest-neighbor classifiers.

Example:

In k-Nearest Neighbors (kNN), Euclidean distance (ℓ_2 norm) determines which training samples are closest to the new input.

2. Parameter Regularization

Norms impose penalties on model parameters to control complexity and reduce overfitting.

Example:

Neural networks apply L2 norm of weights as a penalty added to the loss function:

$$\lambda \|w\|_2^2,$$

encouraging smoother models.

3. Optimization Geometry

Norms define the shape of constraint regions.

- ℓ_2 gives spherical geometry.
- ℓ_1 gives diamond-shaped geometry.
- ℓ_∞ gives cube-shaped geometry.

These shapes influence how gradient-based optimization paths behave.

4. Similarity and Feature Scaling

Feature vectors are often normalized to have unit norm:

$$\hat{v} = \frac{v}{\|v\|}.$$

Example:

Cosine similarity between vectors $\mathbf{x}$ and $\mathbf{y}$ uses normalized vectors:

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

Used in text analysis, recommendation systems, and embedding models.

Concluding Remarks

The norm $\|\mathbf{v}\|$ provides a fundamental quantitative measure of vector magnitude. Its mathematical properties influence loss functions, optimization behavior, regularization strategies, and similarity measures across a wide range of machine-learning algorithms. Different norms emphasize different aspects of data or model structure, making the choice of norm crucial for the performance and interpretability of learning systems.

Data in Machine Learning

- **Machine learning relies on learning patterns from data**
→ ML systems that are fundamentally *data-driven*.
- **Key considerations about data:**
 - **Representation:** How data is structured, encoded, and formatted.
 - **Source:** Where the data originates (sensors, logs, user input, external datasets, etc.).
 - **Preparation:** Cleaning, transforming, and organizing data before feeding it to ML models.

Data types

- **Discrete values:** "yes" / "no", "red" / "white" / "blue"
 - Serve as outputs of **classification**
 - Does not imply ordering
 - Often encoded as integers, but these integers must **not** be used for arithmetic or ordered comparisons.
 - **One-hot encoding** is widely used (e.g., *blue* $\rightarrow [0, 0, 1]$).
- **Integer values:** counts of units
 - It is often useful to convert them to floating points
- **Floating point numbers:**

- Inputs and output of **regression** problems
- We often **normalize** them - bringing their range to [0.0, 1.0] with 0.0 being the minimum value and 1.0 being the maximum

- **Boolean values as floating point**

- It is customary to represent false with 0.0 and true with 1.0
- Intermediate values allow to represent probability, uncertainty, degrees of confidence, etc.

- **Date/Time**

- Should be able to compare it!
- Often represented as an integer number of seconds from a specific moment in time.

Complex data types

- **Fixed length arrays**

- When they are short, you can see them as equivalents of structures in programming languages
- Can be added, scaled

- **Time series**

- Eg. price of a stock on the stock market
- Eg. temperature readings of a patient

Complex data types cont'd

- **Text**

- Usually, variable length list of characters

- **Voice / Sound**

- Usually, variable length list of samples

- **Images**

- Can be seen as an array of pixels (x, y) or a tensor with 3 values per pixel (x, y, color)
- Values usually "normalized" to [0,1].
 - Careful: many external formats represent them as 0...255

- Very often, we need to bring them to the same size before we can do something useful with them...

- **Video**

- Sequences of images

Features

- Often data comes in form of records or rows:
 - e.g. students in the class.
 - one row per student
- Each row contains a set of different data which we call **features**:
 - name, UCF id, date of birth, homework-1-submitted, points, etc.

- We often have some kind of interpretation of the meaning.

• **Creating New Features Through Transformation and Combination**

. **Derived Numerical Features:**

- `Age = floor(current_date - date_of_birth)`

. **Derived Categorical/Boolean Features:**

- `Legally_allowed_to_drive = (age ≥ driving_age_threshold)`

. **Combined or Synthetic Features:**

- `points_in_class × age / ucf_id`
- Demonstrates that feature engineering can produce new variables, though not all will be relevant or beneficial.

Choosing features

- More features are not always better
 - Unnecessary features slow down learning
 - They might confuse the machine learning system
 - Depends on the algorithm
 - At minimum: more features / more data / more work collecting it.
- **Engineering features** used to be a significant part of ML.
- Some ML algorithms can learn which features to pay attention to and which to ignore.
- Other ML algorithms can learn to extract their own features.

Where do you get your data from?

- Collect it from the problem you are studying by careful sampling and measurement.
 - Eg. choose 20 representative Covid patients, and measure their temperature etc.
- Extract from logs
 - Use the hospitals' records of Covid patients and recorded temperatures.
- Big data:
 - If you are Google collect all the messages of type "is a 105F fever means I have Covid" and try to infer how many have indeed Covid etc.

- Quality of data gets worse as the quantity increases.

Public datasets

- There are many publicly available datasets covering various topics
- Some of them can be used to train useful machine learning models
 - But you have to be ready to use the features the creator of the dataset found useful to collect
- Commonly used as learning tools and in competitions
- Others are used to compare and validate datasets
- Kaggle is a website for ML competitions
 - Many publicly available datasets

Private datasets

- For many companies, the data they collected and own are an important part of their business proposition

Preliminary exploration of the data

- Before you dump your data in a machine learning algorithms, **look at it**
- Check maximum / minimum / average values
- Check for unusually high and low values •

What are the data types?

- How many unique values are for a feature? Which are the most popular values?

- Can you make sense of it? Plot a histogram!
- Eg. most popular selling price for a house = 0
 - Why?
- Plot the numerical values in several ways.

Preprocessing and cleaning the data

- Many datasets are imperfect
 - Missing features - no information
 - Wrong / outlier features - with no relationship to true data
 - Noisy features - related to the true data, but with some kind of noise added
- Decisions must be made about what to do with each of these

- Missing features, or data of the wrong type can trigger software failures
- Outlier features (eg house price = 1 trillion dollars) can yield bad results, even if there are few of them

Possible cleaning choices

- Identify outliers
 - For instance, stipulate an acceptable range
- One choice: drop records with missing / outlier data
- Other choice: data imputation
 - Replace the data with an estimated value based on other available information
 - Average of the feature, most frequent value for the feature etc.

- Noise reduction techniques for different data types...
 - Sound, image etc.

What do we want to predict?

- Decide on your machine learning **output**
- Data type, encoding format, range
- Do you have supervised labels for it?

Choosing features

- Often it is advisable to choose the features that enter into your ML algorithm
 - It is surely irrelevant, don't include it.

- Predicting survival by patient id...
- Many algorithms force you to use specific data formats, eg. collections of floats.
- We will say more about this when discussing specific algorithms

Matrix Operations

Matrix operations form the foundation of linear algebra and appear throughout scientific computing, machine learning, quantum computing, control systems, and numerical analysis. Understanding how matrices behave under algebraic operations enables the formal manipulation of linear transformations, vector spaces, and computational models.

Matrix operations not only form the core of linear algebra but also constitute the computational backbone of modern machine learning. Every major algorithm—from linear regression to deep neural networks—relies on structured matrix transformations, enabling efficient computations on high-dimensional data.

1. Matrix Addition and Subtraction

Definition

If A and B are two matrices of the same dimension $m \times n$, their sum and difference are defined elementwise:

$$(A+B)_{ij} = A_{ij} + B_{ij}, (A-B)_{ij} = A_{ij} - B_{ij}$$

Properties

- **Commutative:** $A+B=B+A$
- **Associative:** $A+(B+C)=(A+B)+C$
- **Existence of additive identity:** $A+0=A$
- **Existence of additive inverse:** $A+(-A)=0$

Example

$$A=\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}, B=\begin{pmatrix} 5 & -1 \\ 0 & 7 \end{pmatrix} A+B=\begin{pmatrix} 1+5 & 3-1 \\ 2+0 & 4+7 \end{pmatrix}=\begin{pmatrix} 6 & 2 \\ 2 & 11 \end{pmatrix}$$

Use in Machine Learning

- **Bias updates in neural networks:**

Bias vectors are added to weighted inputs after matrix multiplication:

$$z = Wx + b$$

This operation shifts activation values.

- **Error propagation and gradient updates:**

During training, matrices holding gradients are added/subtracted iteratively:

$$W_{\text{new}} = W_{\text{old}} - \eta \nabla W$$

- **Data augmentation and normalization:**

Performing mean subtraction (data centering) relies on matrix subtraction.

2. Scalar Multiplication

Definition

Multiplying a matrix by a scalar $\alpha \in \mathbb{R}$ yields a matrix whose entries are scaled:

$$(\alpha A)_{ij} = \alpha A_{ij}$$

Example

If

$$A = \begin{pmatrix} 2 & -3 \\ 1 & 0 \end{pmatrix}, \alpha = 3$$

then

$$3A = \begin{pmatrix} 6 & -9 \\ 3 & 0 \end{pmatrix}$$

Scalar multiplication interacts with addition by distributive properties:

$$\alpha(A + B) = \alpha A + \alpha B$$

Use in Machine Learning

- **Learning-rate-scaled gradient descent:**
Gradients are scaled by the learning rate η :

$$W_{\text{new}} = W - \eta \nabla W$$

- **Feature scaling:**
Multiplying feature matrices by scaling factors normalizes feature ranges.
- **Regularization terms:**
L2 regularization:

$$\lambda \|W\|^2 = \lambda W^T W$$

where λ acts as a scalar multiplier.

3. Matrix Multiplication

Definition

If A is an $m \times n$ matrix and B is an $n \times p$ matrix, their product $C = AB$ is an $m \times p$ matrix defined by

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

Matrix multiplication corresponds to composition of linear transformations.

Example

Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, B = \begin{pmatrix} 0 & 1 \\ 5 & 2 \end{pmatrix} \quad AB = \begin{pmatrix} 1 \cdot 0 + 2 \cdot 5 & 1 \cdot 1 + 2 \cdot 2 \\ 3 \cdot 0 + 4 \cdot 5 & 3 \cdot 1 + 4 \cdot 2 \end{pmatrix} = \begin{pmatrix} 10 & 5 \\ 20 & 11 \end{pmatrix}$$

Key Properties

- **Non-commutative:** $AB \neq BA$ in general
- **Associative:** $A(BC) = (AB)C$
- **Distributive:** $A(B+C) = AB + AC$

Use in Machine Learning

- **Forward propagation in neural networks:**

The core computation of each layer:

$$z = Wx$$

where W is the weight matrix.

- **Transformations in linear models:**

In linear regression, predictions are obtained via:

$$\hat{y} = X\beta$$

- **Embeddings and representation learning:**

Multiplying embedding matrices with one-hot encoded vectors retrieves dense embeddings.

- **Attention mechanisms (Transformers):**

Attention = triple matrix multiplication:

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Matrix multiplication is the most computationally expensive and widely used operation in deep learning.

4. Transpose of a Matrix

Definition

The **transpose** of a matrix A , denoted A^T , is formed by swapping rows and columns:

$$(A^T)_{ij} = A_{ji}$$

Example

$$A = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \end{pmatrix} A^T = \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 7 & 8 \end{pmatrix}$$

Properties

- $(A^T)^T = A$
- $(A+B)^T = A^T + B^T$
- $(AB)^T = B^T A^T$

Use in Machine Learning

- **Dot-product attention requires transposes:**

$$Q K^T$$

- **Covariance computation:**

$$\Sigma = \frac{1}{n-1} X^T X$$

- **Backpropagation:**

Transposes appear when propagating gradients backward through fully connected layers:

$$\delta W = \delta z \cdot x^T$$

- **Similarity measurement:**

Dot-product similarity uses vector transposes:

$$u^T v$$

5. Determinant of a Matrix

Definition

The determinant is a scalar associated with a square matrix, representing area/volume scaling of the corresponding linear transformation.

For a 2×2 matrix:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \det(A) = ad - bc$$

Example

$$A = \begin{pmatrix} 3 & 5 \\ 2 & 7 \end{pmatrix} \det(A) = 3 \cdot 7 - 5 \cdot 2 = 21 - 10 = 11$$

Interpretation

- $\det(A) = 0$: matrix is singular; no inverse exists.
- $|\det(A)|$: scaling factor of area/volume.

Use in Machine Learning

- **Normalizing flows:**

Many generative models compute log-determinants of Jacobians to measure how transformations change probability density:

$$\log |\det(J_f)|$$

- **Multivariate Gaussian likelihood:**

The determinant of the covariance matrix appears in:

$$p(x) \propto \frac{1}{\sqrt{\det(2\pi\Sigma)}}$$

- **Checking matrix invertibility:**

Used implicitly when training models requiring full-rank matrices (e.g., certain encoder-decoder transformations).

6. Inverse of a Matrix

Definition

A square matrix A is invertible if there exists A^{-1} such that

$$A A^{-1} = A^{-1} A = I$$

2×2 Case

$$A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Example

$$A = \begin{pmatrix} 4 & 7 \\ 2 & 6 \end{pmatrix} \det(A) = 4 \cdot 6 - 7 \cdot 2 = 10 \quad A^{-1} = \frac{1}{10} \begin{pmatrix} 6 & -7 \\ -2 & 4 \end{pmatrix}$$

Use in Machine Learning

- **Closed-form solution for linear regression:**

$$\beta = (X^T X)^{-1} X^T y$$

- **Kalman filters and state estimation:**
Inverse covariance matrices appear in update equations.
- **Gaussian processes:**
Covariance inversion is required to compute predictive means and variances.
- **Whitening transformation:**

$$X_{\text{white}} = \Sigma^{-1} X$$

Matrix inversion is expensive; ML often approximates or avoids computing it directly.

7. Rank of a Matrix

Definition

The rank is the dimension of the column space (or row space). It measures the number of linearly independent columns.

Example

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$$

Here row 2 = 2 × row 1 → only one independent row → **rank = 1**.

Significance

- Rank determines solution structure of linear systems.
- Full rank implies maximum invertibility properties.

Use in Machine Learning

- **Feature redundancy detection:**
Low rank indicates redundant or linearly dependent features.

- **Dimensionality reduction:**
PCA reveals the intrinsic rank of data through singular values.
 - **Identifiability in linear models:**
 $X^T X$ must be full rank for unique regression solutions.
 - **Latent-factor models:**
Matrix factorization assumes data lies in a low-rank latent space (e.g., recommender systems).
-

8. Dot Product and Matrix–Vector Multiplication

Matrix–vector multiplication extends the dot product:

If $x \in R^n$, and $A \in R^{m \times n}$:

$$Ax = \begin{pmatrix} \langle a_1, x \rangle \\ \langle a_2, x \rangle \\ \vdots \\ \langle a_m, x \rangle \end{pmatrix}$$

where a_i are rows of A .

Example

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, x = \begin{pmatrix} 5 \\ 6 \end{pmatrix} \quad Ax = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}$$

Use in Machine Learning

- **Core computation in neural networks** repeated billions of times.

- **Similarity and distance calculations:**
Cosine similarity, Euclidean distances, projections.
- **Feature extraction:**
Linear transformations of raw data.
- **In SVMs:**
Decision function uses dot products with support vectors:

$$f(x) = \sum_i \alpha_i y_i x_i^T x$$

9. Orthogonality

Definition

Two vectors u, v are orthogonal if $u^T v = 0$.

Matrices with orthogonal columns/rows are **orthogonal matrices**:

$$Q^T Q = I$$

Example

$$Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} Q^T Q = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = I$$

Orthogonal matrices preserve lengths and angles.

Use in Machine Learning

- **Orthogonal weight initialization:**
Prevents vanishing/exploding gradients.
 - **PCA and eigen decomposition:**
Principal components form orthogonal basis vectors.
 - **Orthogonal regularization:**
Improves stability in RNNs, CNNs, GANs.
 - **Distance-preserving transformations:**
Orthogonal matrices preserve norms, crucial in embedding learning.
-

10. Pseudo-Inverse (Moore–Penrose Inverse)

Definition

For a matrix A (not necessarily square), the pseudo-inverse $A^\dagger$ satisfies:

1. $A A^\dagger A = A$
2. $A^\dagger A A^\dagger = A^\dagger$
3. $(A A^\dagger)^T = A A^\dagger$
4. $(A^\dagger A)^T = A^\dagger A$

Use cases

- Solving least-squares problems
- Regression
- Dimension-reduced solutions

Example

For a vector $a = (a_1, a_2, \dots, a_n)^T$,

$$a^\dagger = \frac{a^T}{a^T a}$$

Use in Machine Learning

- **Least-squares solution:**

When $X^T X$ is not invertible:

$$\beta = X^\dagger y$$

- **Linear autoencoders:**

Pseudo-inverse solutions describe optimal linear reconstruction.

- **Dimensionality reduction:**

Used in SVD-based compression and reconstruction tasks.

- **Underdetermined systems:**

Essential in sparse coding or overparameterized neural networks.

11. Eigenvalues and Eigenvectors

Eigenvalues and eigenvectors describe fundamental properties of linear transformations represented by matrices. They reveal invariant directions under transformation, characterize dynamical behavior, and form the basis of matrix decompositions used throughout scientific computing and machine learning.

Let $A \in \mathbb{R}^{n \times n}$ be a square matrix.

A nonzero vector $v \in \mathbb{R}^n$ is an **eigenvector** of A if it satisfies

$$Av = \lambda v$$

where $\lambda \in R$ (or C) is the corresponding **eigenvalue**.

Key Idea

- v is an eigenvector if the action of A **scales** the vector but does not change its direction.
- λ is the **scale factor**.

Eigenvectors are not unique; any scalar multiple of an eigenvector is also an eigenvector.

Example

Let

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

Characteristic polynomial:

$$\det(A - \lambda I) = (2 - \lambda)(3 - \lambda) = 0$$

Eigenvalues: $\lambda_1 = 2$, $\lambda_2 = 3$.

Eigenvector for $\lambda = 2$:

$$(A - 2I)v = 0 \Rightarrow \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} v = 0 \Rightarrow v = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Geometric Interpretation

Eigenvalues and eigenvectors describe how matrices act as **linear transformations** in space.

For a matrix A :

- Each eigenvector is a direction that **A does not rotate**.
- The eigenvalue λ scales the eigenvector by stretching or shrinking it.

Cases:

- $\lambda > 1$: vector stretches
- $0 < \lambda < 1$: vector shrinks
- $\lambda < 0$: vector flips direction
- $\lambda = 0$: vector collapses to zero
- **Complex eigenvalues**: transformation involves rotation or scaling in the complex plane

Example in 2D

A shear matrix has one eigenvalue equal to 1 with a corresponding eigenvector pointing along the unchanged direction.

Algebraic and Geometric Multiplicity

For a given eigenvalue:

Algebraic Multiplicity (AM)

- The number of times λ appears as a root of the characteristic polynomial.

Geometric Multiplicity (GM)

- The number of linearly independent eigenvectors associated with λ .
- This equals the dimension of the nullspace of $A - \lambda I$.

Fundamental Fact

$$1 \leq \text{GM}(\lambda) \leq \text{AM}(\lambda)$$

Matrices that do not have a full set of eigenvectors are called **defective**.

Diagonalization

A matrix A is diagonalizable if and only if it has n linearly independent eigenvectors.

If so, we can decompose:

$$A = P D P^{-1}$$

where:

- D is a diagonal matrix containing eigenvalues,
- P is a matrix whose columns are the eigenvectors.

Why diagonalization matters:

Powers become simple:

$$A^k = P D^k P^{-1}$$

Useful in:

- Markov chains
 - Differential equations
 - Time series prediction
 - Spectral methods
-

Symmetric Matrices and Spectral Theorem

For symmetric matrices ($A = A^T$):

- All eigenvalues are **real**
- Eigenvectors corresponding to distinct eigenvalues are **orthogonal**
- The matrix is **always diagonalizable**
- There exists an orthogonal matrix Q such that:

$$A = Q \Lambda Q^T$$

This is central to PCA, covariance analysis, and kernel methods.

Computing Eigenvalues: Methods

Closed-form formulas

Available only for matrices up to 4×4.

Iterative numerical methods

Used in machine learning and scientific computing:

- **Power iteration**
- **QR algorithm**
- **Lanczos algorithm (for large sparse matrices)**
- **Singular Value Decomposition (SVD)** indirectly yields eigenvalues of $A^T A$

Large models rely heavily on these methods for decompositions.

Stability Interpretation

Eigenvalues determine the stability of linear dynamical systems:

$$x_{t+1} = A x_t$$

- If all eigenvalues satisfy $|\lambda| < 1$: system converges
- If any eigenvalue satisfies $|\lambda| > 1$: system diverges
- If $|\lambda| = 1$: oscillatory behavior

This is important in analyzing:

- RNN stability
- Gradient explosion/vanishing
- Optimization convergence

Applications in Machine Learning

Eigenvalues and eigenvectors have essential roles in several ML algorithms:

1. Principal Component Analysis (PCA)

Diagonalization of covariance matrix:

$$\Sigma = \frac{1}{n} X^T X$$

Eigenvectors $\rightarrow$ principal components

Eigenvalues $\rightarrow$ variance explained

2. Spectral Clustering

Uses eigenvalues of graph Laplacian:

$$L = D - A$$

Eigenvectors represent relaxed cluster indicators.

3. Kernel PCA and manifold learning

Eigenvectors of kernel matrices represent nonlinear principal components.

4. Gaussian Processes

Eigen-decomposition of covariance kernels improves scalability.

5. Markov Models and PageRank

Steady-state distributions correspond to eigenvectors of transition matrices with $\lambda=1$.

6. Neural Networks

- Jacobian eigenvalues describe gradient explosion/vanishing.
- Orthogonality-based regularizations rely on eigen-structure.

7. Matrix Factorization in Recommender Systems

Low-rank approximation uses eigen-decomposition or SVD.

Example: PCA Using Eigenvectors

Given data matrix $X \in R^{n \times d}$:

1. Center data: $X_c = X - \mu$
2. Compute covariance:

$$C = \frac{1}{n} X_c^T X_c$$

3. Compute eigen-decomposition:

$$C = Q \Lambda Q^T$$

4. Use top *keigen*vectors to project data onto lower-dimensional subspace.

Eigenvalues determine which features retain most variance.

Eigenvalues and eigenvectors describe intrinsic geometric and structural properties of linear transformations. They:

- Reveal invariant directions,
- Characterize system stability,
- Enable decomposition and dimensionality reduction,
- Support spectral algorithms used widely in machine learning.

They form the mathematical foundation behind PCA, spectral clustering, Gaussian processes, and many optimization and stability analyses.

- **Principal Component Analysis (PCA):**
Largest eigenvalues correspond to directions of greatest variance.
 - **Spectral clustering:**
Eigenvectors of Laplacian matrices separate clusters.
 - **Stability of dynamic systems:**
Eigenvalues determine stability of RNNs and optimization dynamics.
 - **Covariance structure interpretation:**
Eigen decomposition reveals correlation structure of features.
-

Conclusion

Matrix operations underpin many fundamental structures in computational science. Their algebraic properties allow systematic manipulation of linear systems, statistical models, signal-processing pipelines, and transformations used in physics and machine learning. Mastery of operations such as addition, multiplication, inversion, decomposition, and pseudo-inversion enables efficient modeling and theoretical understanding across disciplines.

Matrix operations provide structural tools that make machine learning algorithms mathematically sound and computationally feasible. Every major algorithm—from regression to neural networks, from PCA to Transformers—depends on linear algebra at its core. A strong grasp of these operations enables deeper understanding of algorithm behavior, optimization dynamics, and model expressiveness.

Pseudo-inverse of matrix

The pseudo-inverse, or **Moore Penrose inverse**, of a matrix A is a generalized inverse that extends the concept of matrix inversion to non-square or rank-deficient matrices. For a matrix $A \in R^{m \times n}$, the pseudo-inverse $A^\dagger \in R^{n \times m}$ satisfies a set of four algebraic conditions that uniquely determine it, even when A does not have a true inverse.

Use in Machine Learning

The pseudo-inverse plays an essential role in:

1. **Linear Regression (Least Squares Solution)**

The closed-form solution to linear regression,

$$\hat{\beta} = (X^\top X)^{-1} X^\top y,$$

can be rewritten as

$$\hat{\beta} = X^\dagger y,$$

which remains valid even when $X^\top X$ is not invertible (multicollinearity, rank deficiency).

2. **Dimension Reduction and Low-Rank Approximation**

When computing optimal low-rank reconstructions via Singular Value Decomposition (SVD), the pseudo-inverse is used to derive projection operators.

3. **Solving Underdetermined or Overdetermined Systems**

Machine-learning algorithms frequently rely on minimizing squared errors in systems with no exact solution, for example, in autoencoders, matrix factorization, and collaborative filtering.

4. Regularization and Stability

Pseudo-inverses derived via SVD expose singular values and allow control of ill-conditioned systems by truncating small singular values (related to Tikhonov/ridge regularization).

Computation via SVD

For any matrix

$$A = U \Sigma V^T,$$

its pseudo-inverse is

$$A^\dagger = V \Sigma^\dagger U^T,$$

where

- U and V contain left and right singular vectors,
 - Σ is a diagonal matrix of singular values σ_i ,
 - $\Sigma^\dagger$ replaces each non-zero σ_i with $1/\sigma_i$ and transposes the matrix.
-

Numerical Example

Consider a non-square matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix}.$$

This matrix is **rank-deficient** because the second row is a multiple of the first.

Step 1: Compute SVD of A

The SVD is

$$A = U \Sigma V^{\top},$$

with

$$\Sigma = \begin{bmatrix} \sqrt{40} & 0 \\ 0 & 0 \end{bmatrix}, U = \begin{bmatrix} 1/\sqrt{10} & -3/\sqrt{10} \\ 3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix}, V = \begin{bmatrix} 1/\sqrt{5} & -2/\sqrt{5} \\ 2/\sqrt{5} & 1/\sqrt{5} \end{bmatrix}.$$

Only one singular value is non-zero: $\sigma_1 = \sqrt{40}$.

Step 2: Construct $\Sigma^{\dagger}$

$$\Sigma^{\dagger} = \begin{bmatrix} 1/\sqrt{40} & 0 \\ 0 & 0 \end{bmatrix}.$$

Step 3: Compute $A^{\dagger}$

$$A^{\dagger} = V \Sigma^{\dagger} U^{\top} = \begin{bmatrix} 1/10 & 3/10 \\ 1/20 & 3/20 \end{bmatrix}.$$

Thus,

$$A^\dagger = \begin{bmatrix} 0.1 & 0.3 \\ 0.05 & 0.15 \end{bmatrix}$$

Using the Pseudo-Inverse to Solve a Least-Squares Problem

Suppose the system

$$Ax \approx \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

has no exact solution because A is rank-deficient.

The least-squares solution is

$$x^\dagger = A^\dagger b.$$

Using the computed pseudo-inverse:

$$x^\dagger = \begin{bmatrix} 0.1 & 0.3 \\ 0.05 & 0.15 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.35 \end{bmatrix}.$$

This solution minimizes the squared error

$$\|Ax - b\|_2.$$

Summary

The pseudo-inverse $A^\dagger$ provides a robust method to solve linear systems, especially when matrices are non-square or singular. In machine learning, it forms the backbone of closed-form regression, linear estimation, matrix reconstruction, and algorithms involving least-squares optimization.

Moore–Penrose Pseudo-Inverse of a Matrix

Full Rank Conditions

Full Row Rank

A matrix of order $m \times n$ has *full row rank* when its rank is equal to the number of rows:

$$\text{rank}(A) = m.$$

This means all rows are linearly independent.

Full Column Rank

A matrix of order $m \times n$ has *full column rank* when its rank is equal to the number of columns:

$$\text{rank}(A) = n.$$

This means all columns are linearly independent.

Rank Deficient

A matrix is *rank deficient* when:

$$\text{rank}(A) < \min(m, n),$$

indicating that its rows and/or columns are not linearly independent.

Moore–Penrose Pseudo-Inverse

The pseudo-inverse $A^\dagger$ generalizes the matrix inverse to rectangular and rank-deficient matrices. It provides a unique solution satisfying the four Penrose conditions and is widely used in least-squares estimation, regression, optimization, and signal processing.

For an $m \times n$ matrix A , the form of the pseudo-inverse depends on rank conditions.

1. Full Column Rank Case

If the columns of A are linearly independent ($\text{rank}(A) = n$), the pseudo-inverse is:

$$A^\dagger = (A^T A)^{-1} A^T.$$

2. Full Row Rank Case

If the rows of A are linearly independent ($\text{rank}(A) = m$), the pseudo-inverse is:

$$A^\dagger = A^T (A A^T)^{-1}.$$

3. Rank-Deficient Case (General Case)

If A is neither full row rank nor full column rank, the pseudo-inverse is computed using Singular Value Decomposition (SVD):

$$A = U \Sigma V^T, A^\dagger = V \Sigma^\dagger U^T.$$

Here, $\Sigma^\dagger$ replaces each non-zero singular value σ by $\frac{1}{\sigma}$.

Worked Examples

Example 1:

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 4 & 3 & 2 \end{bmatrix}$$

Matrix has full row rank ($\text{rank}(A) = 2$), so:

$$A^\dagger = A^T (A A^T)^{-1}.$$

Compute:

$$A A^T = \begin{bmatrix} 6 & 11 \\ 11 & 29 \end{bmatrix}, (A A^T)^{-1} = \frac{1}{5} \begin{bmatrix} 29 & -11 \\ -11 & 6 \end{bmatrix}.$$

Then:

$$A^\dagger = A^T (A A^T)^{-1} = \frac{1}{5} \begin{bmatrix} 1 & 4 \\ 2 & 3 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 29 & -11 \\ -11 & 6 \end{bmatrix}.$$

Final result:

$$A^\dagger = \begin{bmatrix} 1 & -1 \\ 0.2 & 0.2 \\ -0.4 & 0.2 \end{bmatrix}.$$

Example 2:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 1 & -1 \end{bmatrix}$$

Matrix has full column rank ($\text{rank}(A)=2$), so:

$$A^\dagger = (A^T A)^{-1} A^T.$$

Compute:

$$A^T A = \begin{bmatrix} 6 & 3 \\ 3 & 6 \end{bmatrix}, (A^T A)^{-1} = \frac{1}{27} \begin{bmatrix} 6 & -3 \\ -3 & 6 \end{bmatrix}.$$

Thus:

$$A^\dagger = \frac{1}{27} \begin{bmatrix} 6 & -3 \\ -3 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & -1 \end{bmatrix}.$$

Final:

$$A^\dagger = \begin{bmatrix} -2/9 & 2/9 & 1/9 \\ 1/9 & 2/9 & -2/9 \end{bmatrix}.$$

Example 3 (Rank Deficient):

$$A = \begin{bmatrix} 2 & 4 & 3 \\ 1 & 2 & 6 \end{bmatrix}$$

Matrix rank $\text{rank}(A) = 1$, so SVD is required:

$$A^\dagger = V \Sigma^\dagger U^T.$$

Compute $A^T A$, find eigenvalues, eigenvectors, singular values, and normalized singular vectors. After computing SVD:

$$A^\dagger = \begin{bmatrix} \frac{1}{\sqrt{14}} & \frac{2}{\sqrt{14}} \\ \frac{3}{\sqrt{14}} & \frac{2}{\sqrt{14}} \end{bmatrix} \frac{1}{\sqrt{70}} \begin{bmatrix} \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix}.$$

Final pseudo-inverse:

$$A^\dagger = \frac{1}{70} \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}.$$

Summary

The Moore–Penrose pseudo-inverse:

- Extends the inverse to non-square or rank-deficient matrices.
- Provides least-squares solutions in regression and optimization.
- Adapts to the rank structure of the matrix:
 - Full column rank $\rightarrow (A^T A)^{-1} A^T$
 - Full row rank $\rightarrow A^T (A A^T)^{-1}$
 - Rank deficient $\rightarrow$ SVD-based formula $A^\dagger = V \Sigma^\dagger U^T$

The examples illustrate each scenario systematically.

1. Core Thesis

The paper argues that **Large Language Models (LLMs) are not epistemic agents**, even though they often appear to reason or judge like humans. Their outputs are the result of **statistical pattern completion**, not genuine understanding, belief formation, or reasoning. This creates a structural condition called **Epistemia**, where *linguistic plausibility replaces epistemic justification*.

Epistemic agency is a conceptual tool for creating effective teaching methods that involve students taking accountability for their own learning and aligns with the broader emphasis on agency in modern education.

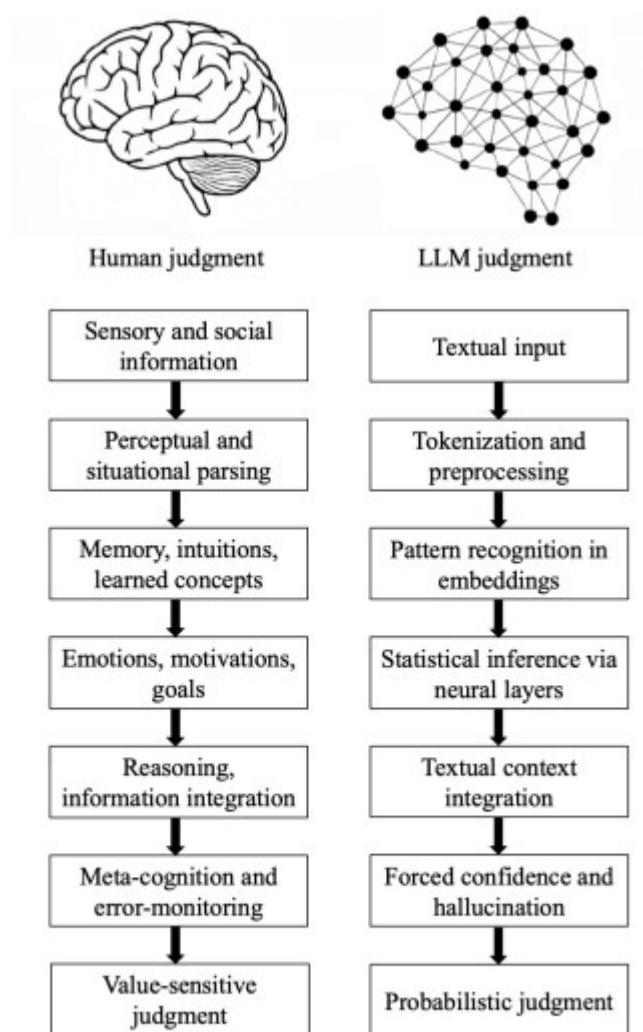


FIG. 1. The human and LLM epistemic pipelines, each organized into seven corresponding stages.

2. Central Claim

LLMs exhibit **surface-level alignment** with human reasoning but **deep epistemic divergence**. They produce fluent and convincing outputs without possessing:

- beliefs
- understanding
- causal reasoning
- epistemic responsibility

As a result, users experience the *illusion of knowing* without engaging in genuine judgment.

3. Core Concept: Epistemia

Epistemia is defined as:

A structural condition in which linguistic plausibility substitutes for epistemic evaluation.

Key properties:

- Answers appear authoritative without being grounded.
- Confidence replaces justification.
- Users receive conclusions without epistemic labor.
- Truth is replaced by statistical plausibility.

Epistemia is not user error or bias; it is **architecturally produced** by generative AI systems.

4. Seven Epistemological Fault Lines

The paper identifies **seven structural divergences** between human cognition and LLMs:

1. Grounding Fault

- **Humans:** Perceive the world through embodied, social, and sensory experience.
- **LLMs:** Operate only on text; meaning is reconstructed statistically.

2. Parsing Fault

- **Humans:** Parse situations holistically using perception and context.
- **LLMs:** Tokenize text mechanically, without semantic understanding.

3. Experience Fault

- **Humans:** Use episodic memory, intuitive physics, and social understanding.
- **LLMs:** Use statistical correlations in embeddings; no lived experience.

4. Motivation Fault

- **Humans:** Guided by emotions, goals, values, and survival motives.
- **LLMs:** Have no goals or values; only optimize probability distributions.

5. Causality Fault

- **Humans:** Reason using causal models and counterfactuals.
- **LLMs:** Rely on correlation, not causal understanding.

6. Metacognitive Fault

- **Humans:** Can detect uncertainty, errors, and withhold judgment.
- **LLMs:** Cannot know when they don't know; hallucination is structural.

7. Value Fault

- **Humans:** Judgments involve responsibility, ethics, and consequences.
 - **LLMs:** Outputs are probabilistic continuations without moral grounding.
-

5. Key Insight: Why LLMs Appear Intelligent

LLMs succeed because:

- They operate in high-probability regions of language.
- Fluency and confidence trigger human credibility heuristics.
- Users mistake linguistic coherence for epistemic reliability.

This creates a **false sense of understanding**, even when content is incorrect or unjustified.

6. Text Generation as a Statistical Process

- LLMs generate text as **stochastic walks over high-dimensional probability graphs**.
- There is no internal notion of truth, reference, or belief.
- “Hallucinations” are not errors but expected outcomes.
- Scaling improves fluency, not epistemic grounding.

7. Implications

A. Evaluation

- Standard benchmarks measure surface correctness, not epistemic validity.
- Need evaluation methods that test uncertainty handling, abstention, and causal reasoning.

B. Governance

- Risk lies not only in harmful outputs but in replacing human judgment.
- Generative systems should not be treated as epistemic authorities.
- Governance must focus on **how outputs are used**, not just what they say.

C. Epistemic Literacy

A new literacy is required to:

- Recognize when AI outputs simulate judgment.
 - Distinguish plausibility from justification.
 - Maintain human accountability in decision-making.
-

8. Core Conclusion

LLMs do not “know” anything in the human sense.
They generate **plausible text without epistemic grounding**.

As systems become more fluent, the danger increases, not because they are wrong more often, but because they *appear right without being epistemically accountable*.

This creates a new epistemic condition; **Epistemia**, where knowledge is replaced by linguistic performance.

One-Hot Encoding (OHE)

One-Hot Encoding represents a categorical variable with K unique classes as a **binary vector of dimension K** where exactly one index is 1 and the rest are 0:

$$x_i = [0, 0, \dots, 1, \dots, 0] \in \{0, 1\}^K$$

Properties

- **Sparse and high-dimensional:** most elements are 0.
- **No ordinal bias:** treats all categories as equidistant.
- **Deterministic & non-learned:** does not capture semantic relationships.
- **Memory-inefficient for large K** but fast for small vocabularies.

Example

Categories: {Red, Green, Blue}

Category	One-Hot Vector
Red	[1, 0, 0]
Green	[0, 1, 0]
Blue	[0, 0, 1]

For labels {Cat, Dog, Fish, Bird} $\rightarrow$ Dog = [0,1,0,0]

Usage in ML

- Input representation for **classical ML models** (Logistic Regression, SVM, Decision Trees when split on categories, Naïve Bayes, etc.).
- Target representation for **multi-class classification** (softmax output layer trained via cross-entropy).
- Acts as a baseline before switching to embeddings when K is large.

Limitations

- **No similarity awareness:** "Cat" and "Dog" are as distant as "Cat" and "Car".
- **Curse of dimensionality** for large category sets (e.g., vocabulary words, code tokens).
- Leads to **unstable or slow learning** in neural models due to lack of continuous geometry.

Transition to Embeddings

OHE inspired learned vector representations such as **Word2Vec**, **FastText**, **Node2Vec**, **Code2Vec**, **CodeBERT**, where the hard binary basis vector is replaced by **trainable dense vectors** that preserve similarity and structure.

Key Takeaway

OHE enables unbiased categorical representation but lacks relational structure; modern ML advances by replacing it with **learned dense embeddings** to support semantic similarity, smooth decision surfaces, and scalable learning.

1. Neural Language Modeling Era (2000–2012)

- **2003 – Bengio Neural Language Model**
First major model to learn **dense word vectors** jointly while training a language model.
Transition: Showed that words can be represented as continuous vectors instead of one-hot encodings.
-

2. Efficient Word Embedding Revolution (2013–2015)

- **2013 – Word2Vec (Mikolov et al.)**
Introduced **Skip-gram & CBOW**, negative sampling, and fast self-supervised training on large corpora.
Influence: Made embeddings scalable and popular for similarity tasks.
 - **2014 – GloVe**
Used **global co-occurrence matrix factorization** rather than local sliding windows.
Transition: Showed statistical signals can also produce strong word embeddings.
 - **2015 – FastText**
Added **subword (character n-gram) modeling** to Word2Vec, improving rare/out-of-vocabulary words.
Influence: Inspired token-level and substructure embeddings beyond natural language.
-

3. Sequence & Sentence-Level Representation (2014–2018)

- **2014 – Doc2Vec**
Extended Word2Vec to embed **documents/paragraphs**.
Transition: Introduced idea of learning embeddings for larger structured text units.

- **2018 – ELMo**
First widely adopted **contextual word embedding** using Bi-LSTMs.
Transition: Demonstrated that a word should have different vectors in different contexts.
 - **2018 – BERT**
Introduced **Transformer + masked language modeling**, producing deeper contextual embeddings.
Influence: Became the blueprint for domain-specific X2Vec transformer models (CodeBERT, etc.)
-

4. Graph / Structural Embedding Expansion (2014–2019)

- **2014 – DeepWalk**
First to apply **Skip-gram idea to graphs** using random walks to embed nodes.
Influence: Proved embeddings are not just for text.
 - **2016 – Node2Vec**
Improved DeepWalk with **biased walks (BFS/DFS trade-off)** for better neighborhood capture.
Transition: Became the standard method for node embeddings.
 - **2017 – Graph2Vec**
Moved from nodes → **entire graph embeddings** using Weisfeiler-Lehman subgraph hashing.
Influence: Inspired embedding of whole structures like functions, files, or code graphs.
-

5. Code Representation Embeddings (2017–2021)

- **2017 – Code2Seq**
Represented code using **AST paths** and sequence decoding.
Transition: Showed that syntax structure paths are useful for embeddings.
 - **2018 – Code2Vec**
Simplified Code2Seq by **aggregating AST paths into a single vector** (attention over paths).
Influence: Direct predecessor of many AST-based bug prediction and code similarity works.
 - **2020 – CodeBERT / CuBERT**
Applied **BERT masked modeling to code tokens**.
Transition: Shift from AST paths → token-level transformer embeddings.
 - **2021 – GraphCodeBERT / GraphCode models**
Added **data-flow or program graph structure** on top of code tokens.
Influence: Connected traditional code embeddings with GNN and program graphs — a key direction for defect prediction.
-

6. ML for Software Defect / Structural Features (2019–2025)

- **HoCS / HoCS-inspired shape features (2019–2024+)**
While not a neural embedding model, HoCS introduced **graph shape histograms as feature vectors**, widely used in classical bug prediction — including your work.
Transition: Parallel movement showing **structure can be encoded into fixed vectors**, influencing graph-based ML pipelines.
 - **2022–2024 – GNN-based code embeddings (GGNN, GAT, etc.)**
Deep learning on **program graphs instead of AST paths alone**.
Influence: Complemented Code2Vec lineage with full graph learning.
-

7. Foundation Model & Cross-Domain Era (2022–Present)

- **2022 – UniXcoder / PL-BERT / Code-LLM embeddings**
Unified embeddings across multiple programming languages using transformers.
Transition: Moved toward **general code understanding embeddings**.
 - **2023–Present – StarCoder / CodeGen / StarCoder2 embeddings**
Large-scale **decoder-only foundation code embeddings** for code generation and retrieval.
Influence: Current state-of-the-art for general code vector representation.
-

learning dense representations from raw data for similarity, prediction, or downstream ML tasks.

General Embedding Models (Text / Tokens)

Concept	Learns embedding for
GloVe	Words (global co-occurrence)
FastText	Words + subwords (character n-grams)
Sent2Vec / Doc2Vec	Sentences / Documents
Contextual embeddings (ELMo, BERT, RoBERTa, etc.)	Words with context-aware vectors
Transformer-based embeddings (Sentence-BERT, OpenAI embeddings, etc.)	Sentences / paragraphs / queries

Code / Program Representation Embeddings

Concept	Embeds
Code2Vec	AST paths of code
Code2Seq	AST paths + sequence generation
CodeBERT	Code tokens using Transformer
GraphCodeBERT	Code tokens + data-flow graph
PL-BERT / UniXcoder / StarCoder embeddings	Multilingual code understanding
CuBERT	Code tokens (like BERT for code)
src2vec / token2vec (various works)	Raw source tokens
AST2Vec	AST nodes
GGNN / GNN-based code embeddings	Program graphs
HoCS / HoCC / HoCS-inspired embeddings	Shape or structural histograms from graphs (<i>relevant to your HoCS feature work</i>)

Graph / Network / Structural Embeddings

These extend the “X2Vec” idea to graphs:

Concept	Embeds
Node2Vec	Graph nodes
DeepWalk	Graph nodes (random walk)
Graph2Vec	Whole graphs
LINE	Large graph nodes
Struc2Vec	Structural similarity of nodes
Sub2Vec	Subgraphs
Diff2Vec	Graph differences

Other Domain-Specific “X2Vec” Models

Concept	Embeds
Image2Vec / CNN feature embeddings	Images
Face2Vec / DeepFace embeddings	Human faces
Audio2Vec / VGGish / Wav2Vec	Audio / speech
Mol2Vec / Chem2Vec	Molecules / chemical structures
BioVec / Gene2Vec / Protein2Vec	Biological sequences
User2Vec / Item2Vec	Users / products (recommendation systems)
Paper2Vec / Cite2Vec	Research papers / citation networks
Knowledge Graph embeddings (TransE, RotatE, ComplEx, etc.)	KG entities + relations
URL2Vec / IP2Vec	URLs / network addresses
Session2Vec	User sessions

Core Shared Idea

All of them learn:

Object → Dense Vector Representation

Using context like:

- co-occurrence (**Word2Vec**, GloVe, Item2Vec)
- program structure (**Code2Vec**, AST2Vec, **GraphCodeBERT**)
- graph neighborhood or structure (Node2Vec, DeepWalk)
- sequence context (**CodeBERT**, **BERT**, Wav2Vec)

Role of Embeddings in Advancing Machine Learning

1. Dimensionality Reduction & Dense Representation

Embeddings convert sparse, high-dimensional one-hot or symbolic inputs into **low-dimensional dense vectors**, reducing computational cost, stabilizing gradients, and improving generalization.

2. Learning Semantic and Structural Similarity

They map similar objects (words, code, graphs, etc.) to **nearby points in vector space**, enabling similarity-based learning, clustering, k-NN search, anomaly detection, and recommendation models.

3. Enabling Self-Supervised Learning at Scale

Models like Word2Vec, FastText, Code2Vec, DeepWalk, and transformer-based encoders learn embeddings from **unlabeled raw data**, allowing ML systems to scale without expensive annotations.

4. Transfer Learning & Feature Reusability

Pretrained embeddings act as **universal feature extractors** that can be reused across tasks, reducing data requirements and accelerating training for downstream classifiers, regressors, or sequence models.

5. Improving Model Input Geometry

Embeddings create **continuous input manifolds**, allowing ML models (SVM, neural nets, GNNs, kernel models) to learn smoother decision boundaries compared to discrete symbolic spaces.

6. Supporting Graph, Sequence, and Multimodal Learning

They extend ML from text to:

- **program structure (Code2Vec, GraphCodeBERT)** → bug prediction, code retrieval
- **nodes and graphs (Node2Vec, Graph2Vec, GNN embeddings)** → network analysis, molecular ML
- **audio (Wav2Vec), images (CNN feature embeddings), multimodal transformers** → cross-domain learning

7. Efficient Retrieval & Scalable Memory

Embedding spaces support **vector databases**, approximate nearest neighbor (ANN) search, FAISS/HNSW indexing, semantic retrieval, and long-term memory systems used in modern ML pipelines.

8. Regularization through Distributed Representations

Information is spread across vector dimensions, acting as an **implicit regularizer**, reducing overfitting and improving robustness to noise and missing tokens.

9. Advancing Defect Prediction & ML on Code

Structural embeddings and graph vectors help ML models capture **syntax, control flow, data dependencies, and geometric properties**, forming the basis for both classical and deep defect prediction systems (including your HoCS + quantum ML research direction).

10. Compatibility with Advanced ML Paradigms

Embeddings integrate naturally with:

- **Neural networks** (stable dense inputs)
- **GNNs** (node/graph embeddings as initial or learned states)
- **Kernel ML** (e.g., RBF, cosine, and quantum kernels operate on embedding vectors)
- **Contrastive learning** (InfoNCE, SimCLR, code similarity training)
- **LLM retrieval embeddings** (StarCoder, UniXcoder, etc.)

- **Quantum ML** (embedding vectors can be encoded into **quantum feature space for kernel similarity**, aligning with quantum kernel and VQC methods)

Key ML Advancements Enabled

ML Capability Improved	Why Embeddings Help
Generalization	smoother continuous feature space
Scalability	self-supervised, unlabeled training
Efficiency	reduced dimensionality, fast ANN
Cross-task transfer	reusable pretrained features
Similarity learning	semantic proximity in vector space
Graph/Code ML	structure preserved in vectors
Kernel learning	kernels operate on dense vectors
Multimodal ML	unified numeric representation

Full Row Rank (or simply Row Rank): Rank of a matrix of order $m \times n$ is equal to the number of rows in the matrix, then matrix has **Full Row Rank**.

Or

Number of linearly independent rows are equal to rank of the matrix, then the matrix has **Full Row Rank**

Full Column Rank (or simply Column Rank): Rank of a matrix of order $m \times n$ is equal to the number of columns in the matrix, then matrix has **Full Column Rank**.

Or

Number of linearly independent columns are equal to rank of the matrix, then the matrix has **Full Column Rank**.

Rank Deficient: Rank of a matrix of order $m \times n$ is less than the number of rows and columns in the matrix, then matrix is Rank Deficient.

MOORE-PENROSE PSEUDO INVERSE

In mathematics, and in particular linear algebra, a **pseudoinverse** A^+ of a matrix A is a generalization of the inverse matrix. The most widely known type of matrix pseudoinverse is the **Moore–Penrose pseudoinverse**.

A common use of the pseudoinverse is to compute a 'best fit' (least squares) solution to a system of linear equations that lacks a unique solution.

The pseudoinverse is defined and unique for all matrices whose entries are real or complex numbers. Matrix inverse exists for square matrices only. Real world data is not always square. Furthermore, real world data is not always consistent and might contain repetitions. To deal with real world data generalized inverse for rectangular matrix is needed. It can be computed using the singular value decomposition also

Let A be a matrix of order $m \times n$ then the pseudoinverse of A is defined as

1.If the columns of a matrix A are linearly independent, then the pseudo inverse of A is: $A^+ = (A^T A)^{-1} \cdot A^T$

2. If the rows of the matrix are linearly independent, the pseudo inverse of A is:

$$A^+ = A^T \cdot (A A^T)^{-1}$$

3. If A has rank deficient, then the Pseudo inverse of A is defined as

$$A^+ = (U \Sigma V^T)^{-1} = (V^T)^{-1} \Sigma^{-1} U^{-1} = V \Sigma^{-1} U^T$$

$$\text{If } \Sigma = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{bmatrix} \text{ then } \Sigma^{-1} = \begin{bmatrix} \frac{1}{\sigma_1} & 0 & 0 \\ 0 & \frac{1}{\sigma_2} & 0 \end{bmatrix}$$

$$1. \text{Find the pseudo inverse of } A = \begin{bmatrix} 1 & 2 & 1 & 3 \\ 4 & 3 & 2 & 1 \end{bmatrix}$$

Sol:

$$\text{Given } A = \begin{bmatrix} 1 & 2 & 1 & 3 \\ 4 & 3 & 2 & 1 \end{bmatrix}$$

$$\text{Here } \begin{vmatrix} 1 & 2 \\ 4 & 3 \end{vmatrix} = 3 - 8 = -5 \neq 0$$

$$\text{rank}(A) = 2$$

Since for the given matrix the rank is equal to the number of rows therefore the matrix has row rank which implies rows are linearly independent

Then the pseudo inverse of A is $A^+ = A^T (A A^T)^{-1}$

$$A = \begin{bmatrix} 1 & 2 & 1 & 3 \\ 4 & 3 & 2 & 1 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 3 \\ 1 & 2 \\ 3 & 1 \end{bmatrix}$$

$$A A^T = \begin{bmatrix} 1 & 2 & 1 & 3 \\ 4 & 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 3 \\ 1 & 2 \\ 3 & 1 \end{bmatrix} = \begin{bmatrix} 15 & 15 \\ 15 & 30 \end{bmatrix}$$

$$|A A^T| = 15(30 - 15) = 225$$

$$(A A^T)^{-1} = \frac{1}{225} \begin{vmatrix} 30 & -15 \\ -15 & 15 \end{vmatrix} = \begin{bmatrix} 2/15 & -1/15 \\ -1/15 & 1/15 \end{bmatrix}$$

$$A^+ = A^T (A A^T)^{-1} = \begin{bmatrix} 1 & 4 \\ 2 & 3 \\ 1 & 2 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 2/15 & -1/15 \\ -1/15 & 1/15 \end{bmatrix} = \begin{bmatrix} -2/15 & 3/15 \\ 1/15 & 1/15 \\ 0 & 1/15 \\ 5/15 & -2/15 \end{bmatrix}$$

$$= \frac{1}{15} \begin{bmatrix} -2 & 3 \\ 1 & 1 \\ 0 & 1 \\ 5 & -2 \end{bmatrix}$$

$$2. \text{Find the pseudo inverse of } A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 1 & -1 \end{bmatrix}$$

Sol:

$$\text{Given } A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\text{Here } \begin{vmatrix} 1 & 2 \\ 2 & 1 \end{vmatrix} = 1 - 4 = -3 \neq 0$$

$$\text{rank}(A) = 2$$

Since for the given matrix the rank is equal to the number of columns therefore the matrix has column rank which implies columns are linearly independent.

Then the pseudo inverse of A is $A^+ = (A^T A)^{-1} A^T$

$$A^T A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 6 & 3 \\ 3 & 6 \end{bmatrix}$$

$$|A^T A| = 36 - 9 = 27$$

$$(A^T A)^{-1} = \frac{1}{27} \begin{vmatrix} 6 & -3 \\ -3 & 6 \end{vmatrix} = \begin{bmatrix} 2/9 & -1/9 \\ -1/9 & 2/9 \end{bmatrix}$$

$$A^+ = (A^T A)^{-1} A^T = \begin{bmatrix} 2/9 & -1/9 \\ -1/9 & 2/9 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 3/9 & 3/9 \\ 3/9 & 0 & -3/9 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$3. \text{Find the pseudo inverse of } A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$$

Sol:

$$\text{Given } A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$$

Here all minors of order two are zero

$$\therefore \text{rank}(A) = 1$$

Since for the given matrix the rank is not equal to the number of rows or columns therefore it has to be solved by singular value decomposition.

Then the pseudo inverse of A using SVD is $A^+ = V \Sigma^+ U^T = V \Sigma^{-1} U^T$

$$\text{Compute } A^T A = \begin{bmatrix} 5 & 10 & 15 \\ 10 & 20 & 30 \\ 15 & 30 & 45 \end{bmatrix}$$

It has eigen values $\lambda_1 = 70, \lambda_2 = 0, \lambda_3 = 0$ and the corresponding

$$\text{eigen vectors are } \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 3 \\ 6 \\ -5 \end{bmatrix}$$

The singular values of A are

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{70}, \sigma_2 = \sqrt{\lambda_2} = \sqrt{0}, \sigma_3 = \sqrt{\lambda_3} = \sqrt{0}$$

$$\text{Normalized vectors are } v_1 = \begin{bmatrix} 1/\sqrt{14} \\ 2/\sqrt{14} \\ 3/\sqrt{14} \end{bmatrix}, v_2 = \begin{bmatrix} -2/\sqrt{5} \\ 1/\sqrt{5} \\ 0 \end{bmatrix}, v_3 = \begin{bmatrix} 3/\sqrt{70} \\ 6/\sqrt{70} \\ -5/\sqrt{70} \end{bmatrix}$$

$$\text{So Thus } V = \begin{bmatrix} 1/\sqrt{14} & -2/\sqrt{5} & 3/\sqrt{70} \\ 2/\sqrt{14} & 1/\sqrt{5} & 6/\sqrt{70} \\ 3/\sqrt{14} & 0 & -5/\sqrt{70} \end{bmatrix} \text{ and } \Sigma = \begin{bmatrix} \sqrt{70} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{Also we can find } u_1 = \frac{1}{\sigma_1} A v_1 = \begin{bmatrix} 1/\sqrt{5} \\ 2/\sqrt{5} \end{bmatrix} \text{ and } u_2 = \frac{1}{\sigma_2} A v_2 = \begin{bmatrix} -2/\sqrt{5} \\ 1/\sqrt{5} \end{bmatrix}$$

$$U = \begin{bmatrix} 1/\sqrt{5} & -2/\sqrt{5} \\ 2/\sqrt{5} & 1/\sqrt{5} \end{bmatrix}$$

$$\therefore A^+ = V \Sigma^+ U^T$$

$$= \begin{bmatrix} 1/\sqrt{14} & -2/\sqrt{5} & 3/\sqrt{70} \\ 2/\sqrt{14} & 1/\sqrt{5} & 6/\sqrt{70} \\ 3/\sqrt{14} & 0 & -5/\sqrt{70} \end{bmatrix} \begin{bmatrix} 1/\sqrt{70} & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{5} & 2/\sqrt{5} \\ -2/\sqrt{5} & 1/\sqrt{5} \end{bmatrix}$$

$$= \frac{1}{70} \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

Optimizing Word2Vec Performance on Multicore Systems

Vasudevan Rengasamy, Tao-Yang Fu, Wang-Chien Lee, Kamesh Madduri
The Pennsylvania State University



Acknowledgement

This research is supported in part by the US National Science Foundation grants ACI-1253881, CCF-1439057, IIS-1717084, and SMA-1360205. This work used the Extreme Science and Engineering Discovery Environment (XSEDE), which is supported by National Science Foundation grant number ACI-1548562.

Outline

- Introduction
 - Motivation for Word Embeddings
 - Prior work on optimizing **Word2Vec**
- Our Context Combining optimization
- Experiments and Results
 - Parallel performance
 - Evaluating Accuracy
- Conclusion and Future work

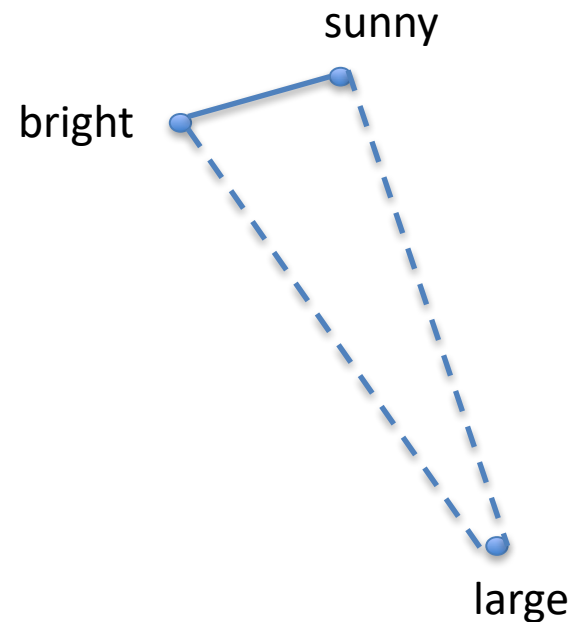
Word embedding

Learn low dimensional vector representation for words

Vectors of similar words are closer in the vector space

Applications:

- Document classification
- Machine translation
- Named entity recognition



Word2Vec: A Word Embedding Technique

Efficient Estimation of Word Representations in Vector Space - arXiv

<https://arxiv.org> > cs ▼

by T Mikolov - 2013 - Cited by 4668 - Related articles

Jan 16, 2013 - We propose two novel model architectures for computing continuous **vector representations** of **words** from very large data sets. The quality of these **representations** is measured in a **word** similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks.

It is a bright sunny day. Sahara is the largest desert.



Two applications of Word2Vec

Word Similarity

Word	Word	Similarity score
football	soccer	0.83
television	radio	0.76
precedent	group	-0.05
listing	proximity	-0.06

Word Analogy

Question	Predicted Word
greece:athens france:?	paris
greece:athens china:?	beijing

Aim

Improve efficiency of Word2Vec training on multi-core systems by

- Improving floating point throughput
- Reducing overheads

Avoid any accuracy loss due to performance optimization

Training SGNS model

1. Select target word, input context

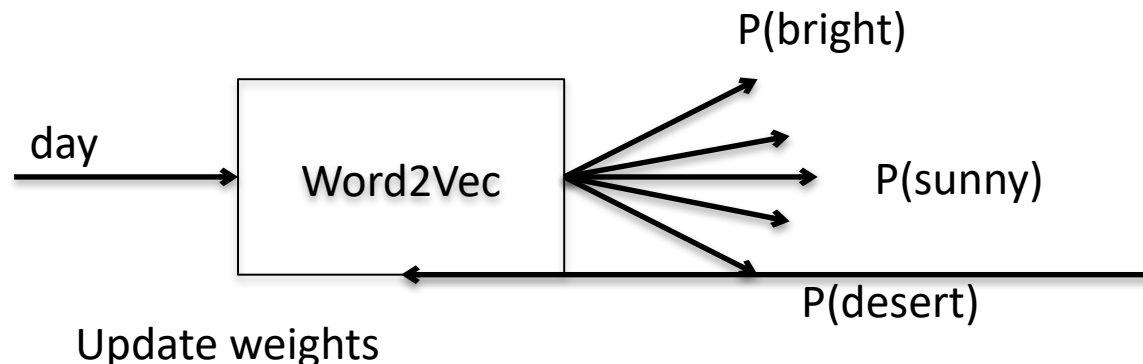
It is a bright sunny day. Sahara is the largest desert.



2. Generate Samples

day, bright, +	desert, bright, -
day, sunny, +	largest, sunny, -

3. Update model parameters using Stochastic Gradient Descent (SGD)



Prior Work

pWord2Vec

Idea: Share negative samples among same context words [2]

Advantages:

- Matrix multiplications which are compute bound

Limitations:

- Small matrix → Less floating point throughput
- Overhead to create dense matrices

It is a bright sunny day. Sahara is the largest desert.

Input Word	Sample word	Label
bright	day	+
bright	desert	-
sunny	day	+
sunny	desert	-

Probability Calculation

$$\text{sig} \left(\begin{bmatrix} \text{day} \\ \text{desert} \end{bmatrix}_{2,D} \begin{bmatrix} \text{bright} \\ \text{sunny} \end{bmatrix}_{D,2}^T \right)$$

Context Combining

Context Combining: Overview

It is a bright sunny day. Sahara is the largest desert.

C1
C2

C1

Input Word	Sample word	Label
bright	day	+
bright	desert	-
sunny	day	+
sunny	desert	-

C2

Input Word	Sample word	Label
largest	desert	+
largest	day	-

Probability calculation: $\text{sig} \left(\begin{bmatrix} \text{day} \\ \text{desert} \end{bmatrix}_{2,D} \begin{bmatrix} \text{bright} \\ \text{sunny} \\ \text{largest} \end{bmatrix}_{D,3}^T \right)$

C1, C2 – Related windows sharing samples

Context Combining: Steps

Algorithm 1 Context Combining algorithm

Input: Training data ts containing T' words, Data segment size T , Number of contexts to combine C , Number of epochs I

- 1: **for** $i = 1$ to I **do**
 $t \leftarrow 1$
- 2: **while** $t < T'$ **do**
- 3: Read $ts[t..t + T]$ words from training data
- 4: Mark words as unprocessed
- 5: Preprocess
- 6: **while** $\exists$ unprocessed target word **do**
- 7: $out \leftarrow$ target word
- 8: Find $C - 1$ related contexts
- 9: Perform SGD updates
- 10: Mark target words of C contexts as processed
- 11: **end while**
- 12: $t \leftarrow t + T$
- 13: **end while**
- 14: **end for**

Preprocessing

Create Inverse index for finding target word positions in training data segment

Used for finding *related* windows that contain a given word as the target

CSR representation used

Training Data segment

It is a bright sunny day. Sahara is the largest desert.

Inverse Index

Target word	It	is	a	bright	Sunny	day	Sahara	the	largest	desert
Offsets	0	1,7	2	3	4	5	6	8	9	10

CSR representation

	It	is	a	bright	sunny	day	Sahara	the	largest	desert	
Index	0	1	3	4	5	6	7	8	9	10	
wOffset	0	1	7	2	3	4	5	6	8	9	10

Experiments and Results

Experimental Setup

System Description

1. Stampede supercomputer at TACC – 2 x 8-core Intel Xeon E5 (Sandy Bridge) processors and 32GB DDR3 memory
2. Stampede2 supercomputer at TACC - 1 68 core Intel Xeon Phi 7250 Knights Landing (KNL) processor with 4 way SMT, 96 GB DDR4, 16 GB MCDRAM

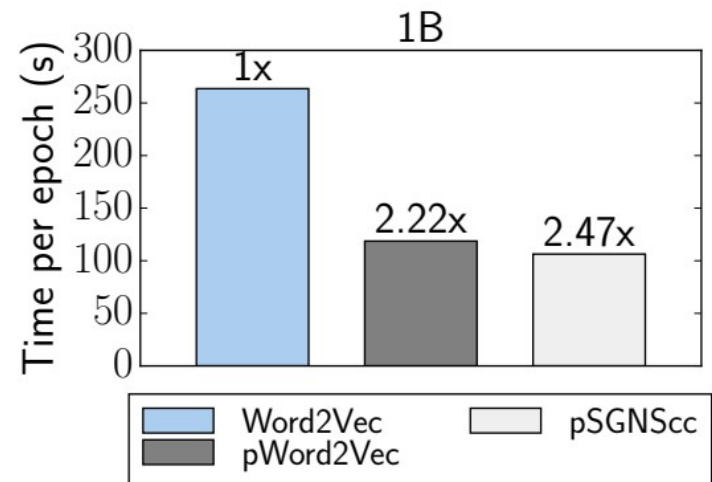
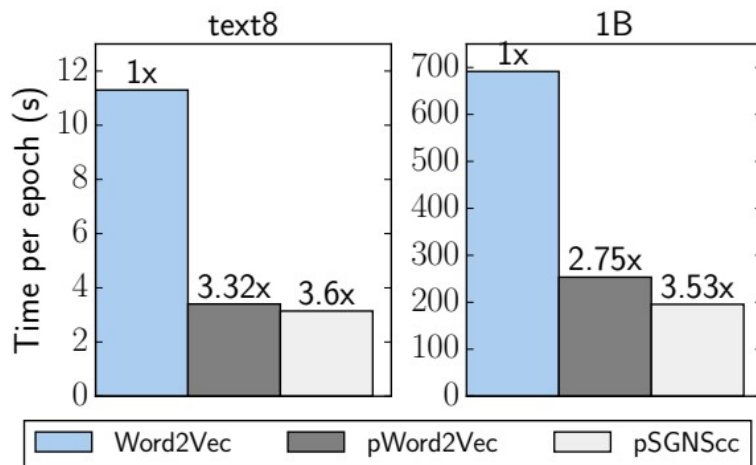
Note: All our experiments were run on a single compute node.

Datasets

Training Data	Test datasets
1. text8 – 17M words, Vocabulary Size – 71K 2. 1 billion word benchmark (1B) – 805M words, Vocabulary size – 1.1M	1. WordSim353 (ws353) for word similarity test 2. Google analogy queries for word analogy tests

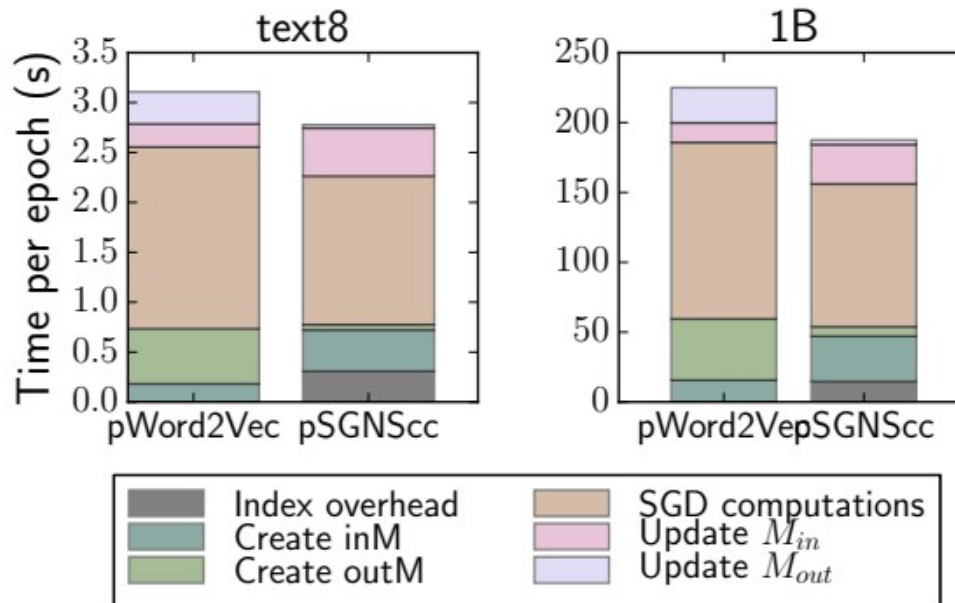
Multi-core performance comparison

Intel Sandy Bridge: 16 threads Intel KNL: 272 threads



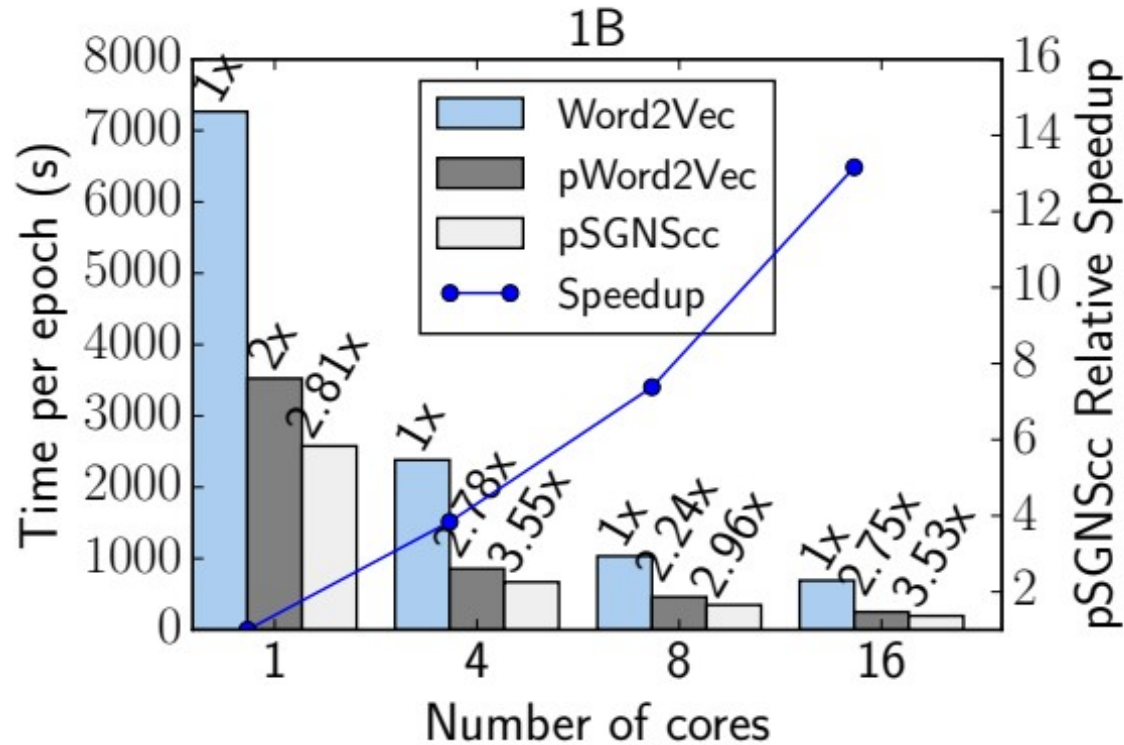
For 1B dataset, pSGNScc achieves 1.28x speedup over pWord2Vec in Stampede and 1.11x speedup in Stampede2

Breakdown of time



- Index overhead, Create inM, Update M_{in} – pWord2Vec is faster
- Create outM, SGD computations, Update M_{out} – pSGNScc is faster

Parallel Scaling



Evaluating Accuracy

Method	Similarity		Analogy	
	text8	1B	text8	1B
Word2Vec	.671	.636	.297	.330
pWord2Vec	.682	.639	.309	.327
pSGNScc	.685	.633	.322	.328

text8 – Accuracy of pWord2Vec and pSGNScc better than Word2Vec

1B – Accuracy of 3 methods are comparable

Performance impact of T and C

	Value of T			
	100K	500K	1M	2M
Time per epoch (s)	220.13	196.03	196.74	201.13
Index time (s)	17.20	14.85	17.97	24.22
Avg. related windows (≤ 8)	5.49	7.53	7.80	7.92

	Value of C			
	1	4	8	16
Time per epoch (s)	258.38	208.45	196.03	191.08
Index time (s)	3.16	12.47	14.92	20.25
SGD Computations (s)	129.94	108.27	102.12	95.69

Conclusion and Future Work

Conclusion

- Proposed new optimization technique called Context Combining for
 - Improving floating point throughput
 - Reducing Overhead
- Speedup of up to 1.28x over pWord2Vec and 3.53x over Google's multi-threaded C implementation
- Accuracy comparable to state-of-the-art implementations

Future Work

- Floating point throughput is still less than peak performance
 - Explore alternate approaches context combining
- Optimize Word2Vec for GPUs
- Support distributed memory parallelism

References

- 1 Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In Proc. Int'l. Conf. on Learning Representations (ICLR) Workshop.
- 2 Shihao Ji, Nadathur Satish, Sheng Li, and Pradeep Dubey. 2016. Parallelizing Word2Vec in Multi-Core and Many-Core Architectures. In Proc. Int'l. Workshop on Efficient Methods for Deep Neural Networks (EMDNN).
- 3 Recht, B., Re, C., Wright, S., & Niu, F. (2011). Hogwild: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in neural information processing systems*.
- 4 Levy, O., Goldberg, Y., & Dagan, I. (2015). Improving distributional similarity with lessons learned from word embeddings. *Transactions of the Association for Computational Linguistics*, 3, 211-225.

Paper doi>[10.1145/3149704.3149768](https://doi.org/10.1145/3149704.3149768)

GitHub Source code : `vasupsu/pWord2Vec.git`

GitHub Artifact : `vasupsu/IA3_Paper16_ArtifactEvaluation.git`

Thank You

Questions?

For questions, Email: `vxr162@psu.edu`

Skip-gram with Negative Sampling

- Skip-gram model is slow due to Softmax function – $O(V)$ computation per sample
- Skip-gram with Negative Sampling (SGNS) model reduces the complexity
 - For each target word, choose K words (typically 5-25) at random from the vocabulary
 - Target word = positive sample
 - K Random words = negative samples

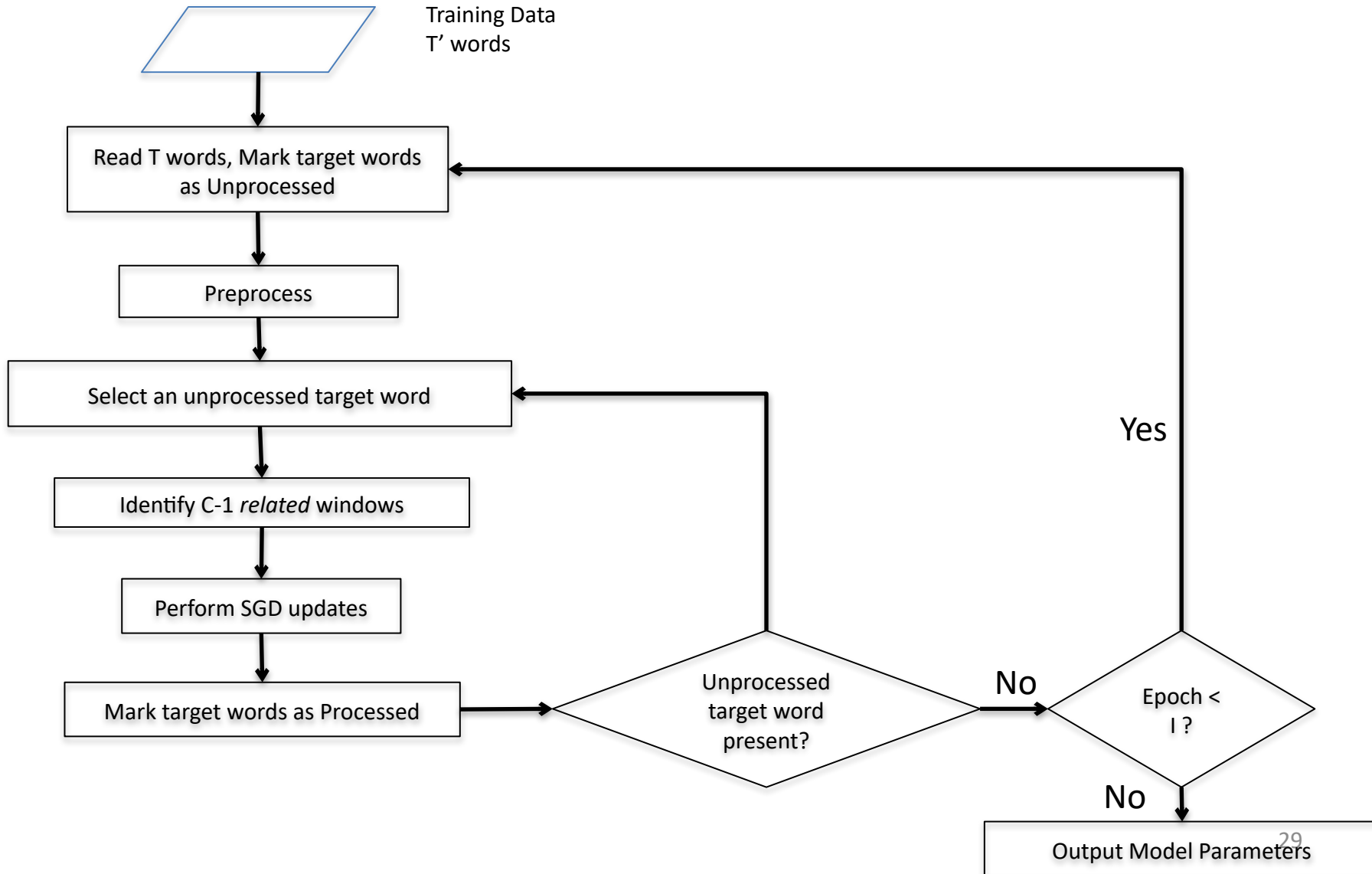
Skip-gram with Negative Sampling

- Probability calculation under SGNS model

$$\log P(y|x) = \log \text{sig}(v_x v_y) + \sum_{i=1}^K \log \text{sig}(-v_x v_i)$$

- Time complexity for processing 1 sample: $O(K)$
- SGNS training: Use SGD to maximize $P(y|x)$ for positive samples and minimize for negative samples
- Parallelism achieved by processing contexts asynchronously (ignoring race conditions)

Context Combining: Steps



Skip-gram with Negative Sampling

It is a bright sunny day. Sahara is the largest desert.

└──────────┘

Input context



Target word

Input Word	Sample word	Label	Probability Calculation
bright	day	+	
bright	desert	-	
sunny	day	+	
sunny	largest	-	

sig (<day . bright>)
sig (<desert . bright>)
sig (<day . sunny>)
sig (<largest . sunny>)

Drawback: Vector products are memory-bound